Faculty of Sciences Departement of Computer Science

Master thesis

Canonical Labelling and Graph Isomorphism: A C++ Reimplementation of Nauty

Auteur: Rodolphe Prévot

Promoteur : Robin Petit

Directeur : Gwenaël Joret

Academic year 2025–2026

Acknowledgements

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Contents

Chapter 1

Introduction

Two graphs may look entirely different when drawn on paper, yet represent the exact same underlying structure. Formally deciding whether this is the case is known as the graph isomorphism problem (GI), one of the most studied problems in theoretical computer science. Fully retracing its history is nearly impossible given the sheer number of works devoted to the topic, but we will review its most significant milestones. GI first emerged as a computational problem in the chemical literature in the 1950s [54] (as cited in [29]), where it took the form of a matching problem between a "molecular graph" and graphs of the same type stored in a database. This approach lies at the origins of cheminformatics (the application of computer science to chemistry) and gave rise to some of the first dedicated systems: Stephen H. Unger proposed GIT (Graph Isomorphism Tester) [63], one of the earliest heuristics for testing the isomorphism of pairs of directed line graphs, while Edward H. Sussenguth introduced a graph-theoretic algorithm specifically designed for matching chemical structures [61]. These early domain-specific efforts laid the foundations for the more general algorithmic developments that followed.

Throughout the 20th century, considerable research efforts were devoted to solving GI, with varying degrees of efficiency. On the one hand, one of the simplest approaches is based on backtracking, a widely known and relatively easy-to-implement technique, but it suffers from poor scalability and provides no guarantee of efficiency in the worst case [56]. On the other hand, more sophisticated methods were also proposed, such as the construction of a so-called representative graph and a reordered graph to test for isomorphism [14]. Although this general approach was later shown to be incorrect (as cited in [22]), it played an important role in stimulating further investigations into the graph isomorphism problem.

Finally, a more promising direction emerged: reducing graphs to their canonical form. The idea is straightforward: every graph can be transformed into a *unique* canonical representation [3], meaning that two isomorphic graphs will share the same canonical form. This shift in perspective transformed GI from a problem of structural equivalence into one of equality: rather than directly checking whether two graphs are isomorphic, one simply compares their canonical forms, leading to more efficient solutions.

In 1981, Brendan D. McKay marked a turning point with the introduction of his software *Nauty* (No AUTomorphisms, Yes?) [44], offering a far more efficient approach to tackling GI. The program had actually been started in 1976 under the name GLABC, written in Fortran and Assembly, before being finalised in C in 1981 and renamed *Nauty*. Since then,

the program has undergone continuous optimisation to maximise its speed and efficiency. The key idea behind *Nauty* is to compute a graph's canonical form without relying on matrix representations. Instead, it uses a search tree combined with a partition refinement system to generate the canonical form.

Nowadays, this canonical form capability enables a powerful application: the exhaustive generation of non-isomorphic graphs, that is, given n vertices (and optionally additional constraints), enumerating all graphs that are pairwise non-isomorphic, a problem that researchers have been studying for decades. For instance, the generation of cubic graphs has been investigated since the 1960s, as these graphs are useful in various fields: in chemistry, they can model carbon networks; in mathematics and computer science, they often serve as minimal counterexamples in graph theory [11]. Brendan D. McKay, for his part, successfully connected his tool *Nauty* to the generation of non-isomorphic graphs [43].

Nauty is much more than just a tool for canonical labelling and the generation of non-isomorphic graphs. It has evolved into a comprehensive program supporting a wide range of graph families and offering additional features, such as graph statistics and other advanced analysis tools [47]. Furthermore, libraries based on *Nauty* have been developed for several programming languages: Pynauty in Python [19], NautyGraphs in Julia [20], and even pl-nauty in Prolog [23].

In practice, graph isomorphism tools are highly useful and widely used in scientific research. In general, these tools are employed to detect symmetries in combinatorial objects, which allows them to be applied across a variety of domains. In combinatorial search, for instance, GI tools are used in SAT solving, where symmetries can be exploited to reduce the search space [33]. Another application arises in data collections: for molecular databases, for example, computing the canonical form of a new molecule makes it possible to directly compare it against the database, rather than testing isomorphism against every entry individually. Further applications can also be found in machine learning, software verification, and many other fields (as cited in [29]).

The main goal of this thesis is to re-implement one of the most efficient tools for graph isomorphism, *Nauty*, using a more modern programming language: C++. Recreating *Nauty* from scratch will allow us to evaluate optimisation differences, make its implementation more modern and easier to understand, and gain a deeper understanding of the algorithm itself. Indeed, the original implementation consists of several thousand lines of highly optimised C code, making it difficult to follow at first glance. Furthermore, although McKay's original paper describes the main ideas behind the algorithm, many implementation details are left implicit. Reimplementing *Nauty* therefore requires uncovering these subtleties and making a number of design decisions that are not explicitly documented. The objective of this new codebase is certainly not to achieve better raw performance than the original *Nauty*, but rather to provide a modern perspective on the tool. Moreover, C++ remains a widely used programming language, and a C++ implementation of *Nauty* can make it more accessible to a broader audience while retaining better performance than more popular but interpreted languages such as Python.

This thesis is divided into four main chapters. The first introduces the fundamental concepts and preliminaries necessary to fully understand the key ideas discussed throughout this work. The second presents a comprehensive state of the art, summarising previous research on the graph isomorphism problem. The third is devoted to McKay's canonical labelling algorithm and the construction of the automorphism group. Finally, the fourth chapter presents our C++ implementation of *Nauty* and discusses the results obtained.

Chapter 2

Preliminaries

This chapter introduces all the definitions necessary to understand this document.

2.1 Algebra definitions

Definition 1 (informal)**.** An *isomorphism* is a bijective mapping between two structures that preserves the relationships between their elements. If a mapping sends an element x of one structure to an element y of another structure, we write $x \mapsto y$.

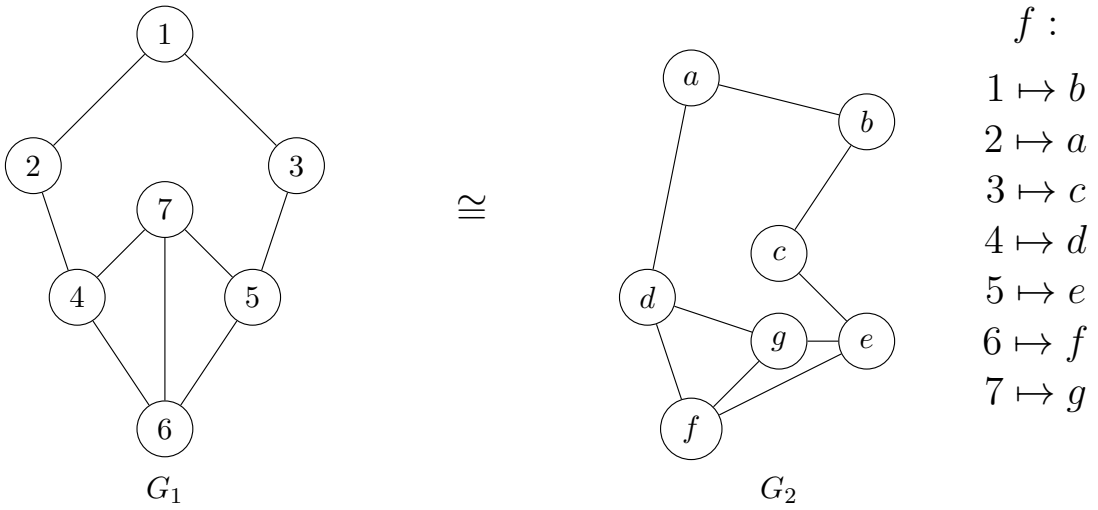


Figure 2.1: Example of isomorphism between two graphs G_1 and G_2

An *automorphism* is an isomorphism from a set to itself.

Definition 2. A *group* is a set with a binary operation that satisfies the following constraints: the operation is associative, it has an identity element, and every element of the set has an inverse. For example, $(\mathbb{R}, +)$ is a group: it consists of the set of real numbers with addition as the binary operation. It has as identity element 0; every element has an inverse element (each number with its opposite); and it is associative: for all numbers a, b, c in $\mathbb{R}$, $a + (b + c) = (a + b) + c$.

Let X be a finite set. The *symmetric group* on X, denoted S_n when $|X| = n$, is the group

of all *permutations* of X, i.e. all bijections from X to itself, with function composition as the group operation.

Let $(G, *)$ be a group with identity element e, and let X be a non-empty set. A *group action* of G on X is a map

$$X \times G \to X : \quad (x, g) \mapsto x^g,$$

where x^g denotes the image of x under g, such that the following two properties hold:

1. identity: $\forall x \in X,\ x^e = x$,
2. compatibility: $\forall g, h \in G,\ \forall x \in X,\ x^{(g*h)} = (x^g)^h$.

As an example, S_n acts on X (when $|X| = n$) in exactly this way: for $x \in X$ and $g \in S_n$, x^g is the image of x under the permutation g, and composing two permutations $g, h \in S_n$ corresponds to applying g first and then h, consistent with $x^{(gh)} = (x^g)^h$.

The same exponent notation is naturally extends to structures derived from X. In particular, for $W \subseteq X$, we write $W^g = \{w^g : w \in W\}$ and similarly for sequences.

Definition 3. Let a group G act on a set X, and let $S \subseteq X$. The *point-wise stabiliser* of S in G is the subgroup defined by:

$$G_{(S)} = \{g \in G \mid \forall x \in S,\ x^g = x\}.$$

Definition 4. Let $G \subseteq S_n$ be a permutation group, acting on $\{1, \ldots, n\}$ via the action of S_n, and let $\alpha \in \{1, \ldots, n\}$. The *orbit* of α under the action of G is the set

$$\mathrm{Orb}_G(\alpha) = \{\alpha^g \mid g \in G\}.$$

Definition 5. Let $G \subseteq S_n$ be a permutation group, acting on $\{1, \ldots, n\}$ via the action of S_n, and let $\alpha \in \{1, \ldots, n\}$. The *stabiliser* of α in G is the subgroup

$$G_\alpha = \{g \in G \mid \alpha^g = \alpha\}.$$

2.2 Graph theory definitions

For further details on the following concepts, refer to Reinhard Diestel's book on graph theory [18].

Definition 6. A *graph* is a pair $G = (V, E)$ of sets satisfying $E \subseteq \binom{V}{2}$; thus, the elements of E are 2-element subsets of V. The elements of V are the *vertices* of the graph G, the elements of E are its *edges*.

Let V^* denote the set of finite sequences of vertices. For $\sigma \in V^*$, $|\sigma|$ denotes the length of σ. If $\sigma = (v_1, v_2, \ldots, v_k) \in V^*$ and $w \in V$ then $\sigma \mid\mid w$ denotes $(v_1, v_2, \ldots, v_k, w)$. Furthermore, for $0 \leq s \leq k$, $[\sigma]_s \coloneqq (v_1, v_2, \ldots, v_s)$.

An acyclic graph (one not containing any cycles) is called a *forest*. A connected forest is called a *tree*. Each vertex of a tree is called a *node*, and a node of degree 1 is called a *leaf*. When a particular node is designated as the starting point, that node is called the *root*. Given a tree T, a *subtree* T' is a connected subgraph of T (that is itself a tree). We write $T' \subseteq T$ to denote that T' is a subtree of T.

Definition 7. We let $\mathcal{G}_n$ be the set of all graphs on n vertices (without loss of generality, say $V = \{1, 2, \ldots, n\}$). $\mathcal{G}$ represents the set of all graphs.

Definition 8. The *star graph* $K_{1,3}$ is a tree with one internal node and three leaves. It is also known as the *claw graph*. The internal node is called the *center* of the star graph. For most of our examples, we will use this graph.

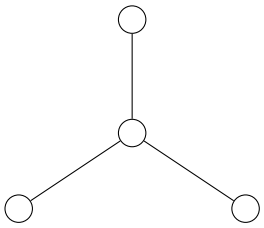


Figure 2.2: The star graph $K_{1,3}$ with its center and leaves.

2.2.1 Definitions specific to isomorphisms

The definitions in this section are specific to the context of *Nauty*, and some terms carry a meaning different from their standard usage elsewhere in graph theory. In particular, a *colouring* here should not be confused with a *proper colouring*: π refers to a *partitioning* of the vertices.

Definition 9. A *colouring* of V (or of $G \in \mathcal{G}$) is a surjective function π from V onto $\{1, 2, \ldots, k\}$ for some k. The number of colours, i.e. k, is denoted by $|\pi|$. A cell of π is the set of vertices with some given colour; that is, $\pi^{-1}(j)$ for some j with $1 \leq j \leq |\pi|$. Equivalently, we may write π as the ordered sequence of its cells, $\pi = (X_1, X_2, \ldots, X_k)$, where $X_j \coloneqq \pi^{-1}(j)$ for $1 \leq j \leq k$; we use this cell-sequence notation interchangeably with the functional notation $\pi(v)$ throughout. We let Π denote the set of all colourings of V.

If π, π' are colourings of V, then π' is *finer than or equal* to π, written $\pi' \preccurlyeq \pi$, if $\pi(v) < \pi(w) \Rightarrow \pi'(v) < \pi'(w)$ for all $v, w \in V$ (This implies that each cell of π' is a subset of a cell of π).

A *discrete colouring* is a colouring in which each cell is a singleton. Therefore, if $G = (V, E)$ with $|V| = n$, then $|\pi| = n$.

A *coloured graph* is a pair (G, π) where G is a graph and π is a colouring of G.

A colouring of a graph is called *equitable* if any two vertices of the same colour are adjacent to the same number of vertices of each colour. This is also called an *equitable/stable partition*.

Definition 10. Let S_n denote the *symmetric group* acting on V and let $g \in S_n$.

There are several notations to consider:

1. if π is a colouring of V, then π^g is the colouring with $\pi^g(v^g) = \pi(v)$ for each $v \in V$.
2. if $G = (V, E)$ is a graph, then $G^g = (V^g, E^g)$ is the graph obtained by applying the permutation g to the vertices, where

$$V^g = \{v^g \mid v \in V\}, \qquad E^g = \{\{u^g, v^g\} \mid \{u, v\} \in E\}.$$

3. if (G, π) is a coloured graph, then $(G, \pi)^g = (G^g, \pi^g)$.

These properties show that two coloured graphs (G, π) and (G', π') are isomorphic iff there exists $g \in S_n$ such that $(G', \pi') = (G, \pi)^g$. In this case, we write $(G, \pi) \cong (G', \pi')$.

$\mathrm{Aut}(G, \pi)$ is the *automorphism group* of the coloured graph (G, π), defined as

$$\mathrm{Aut}(G, \pi) = \{g \in S_n : (G, \pi)^g = (G, \pi)\}.$$

Let $g_1, \ldots, g_k \in \mathrm{Aut}(G, \pi_0)$. The *subgroup generated by* $g_1, \ldots, g_k$, denoted $\langle g_1, \ldots, g_k \rangle$, is the smallest subgroup of $\mathrm{Aut}(G, \pi_0)$ containing $g_1, \ldots, g_k$; equivalently, it is the set of all finite products of the g_i and their inverses.

Definition 11. A *canonical form* is a function

$$C : \mathcal{G} \times \Pi \to \mathcal{G} \times \Pi$$

such that for all $G \in \mathcal{G}, \pi \in \Pi$ and $g \in S_n$:

1. $C(G, \pi) \cong (G, \pi)$.
2. $C(G^g, \pi^g) = C(G, \pi)$.

Informally, the canonical form $C(G, \pi)$ of a coloured graph is an isomorphic representative of the graph that depends only on its isomorphism class. In other words, C maps any two coloured graphs related by a relabelling to exactly the same output. Consequently, two coloured graphs are isomorphic if and only if they have the same canonical form.

2.3 Complexity theory definitions

For more informations on the complexity definitions, you can refer to Michael Sipser's book: Introduction to the Theory of Computation [60].

Definition 12 (minimalist)**.** The class NP (comes from nondeterministic polynomial) is the set of decision problems for which the problem instances have certificates verifiable in polynomial time by a deterministic Turing machine, or alternatively the set of problems that can be solved in polynomial time by a nondeterministic Turing machine.

A decision problem is a member of coNP iff its complement is in the complexity class NP.

A decision problem B is NP-*complete* if it satisfies the two following conditions:

1. B belongs to the class NP.
2. Every problem in NP can be reduced to B in polynomial time. A problem A can be reduced to a problem B in polynomial time if any instance of A can be transformed into an equivalent instance of B using a polynomial-time algorithm.

Definition 13. A problem is NP-*intermediate* (NPI) if it is not NP-complete nor P.

Definition 14. A language L belongs to the class *SPP* (Stoic PP) if there exists a polynomial-time nondeterministic Turing machine M such that

$$x \in L \iff \#\mathrm{acc}_M(x) = 1 \text{ and } x \notin L \iff \#\mathrm{acc}_M(x) = 0,$$

where $\#\mathrm{acc}_M(x)$ denotes the number of accepting computation paths of M on input x.

Definition 15. LOGSPACE (or L) is the complexity class of decision problems that can be solved by a deterministic Turing machine using a logarithmic amount of writable memory.

Definition 16. An *oracle* for a language B is an external device that is capable of reporting whether any string w is a member of B. An oracle Turing machine is a modified Turing machine that has the additional capability of querying an oracle. We write M^B to describe an oracle Turing machine that has an oracle for language B.

The *Polynomial-Time Hierarchy* (PH) is a hierarchy of complexity classes that generalizes the classes NP and coNP. Formally, the base level of the hierarchy is defined as:

$$\Delta_0^{\mathsf{P}} \coloneqq \Sigma_0^{\mathsf{P}} \coloneqq \Pi_0^{\mathsf{P}} \coloneqq \mathsf{P},$$

where P denotes the class of decision problems solvable in polynomial time by a deterministic Turing machine. Then, for every $i \geq 0$, the hierarchy is defined recursively as follows:

- $\Delta_{i+1}^{\mathsf{P}} \coloneqq \mathsf{P}^{\Sigma_i^{\mathsf{P}}}$
- $\Sigma_{i+1}^{\mathsf{P}} \coloneqq \mathsf{NP}^{\Sigma_i^{\mathsf{P}}}$
- $\Pi_{i+1}^{\mathsf{P}} \coloneqq \mathsf{coNP}^{\Sigma_i^{\mathsf{P}}}$

Here, P^A denotes the class of problems solvable in polynomial time by a deterministic Turing machine equipped with an oracle for some complete problem in class A. The classes NP^A and coNP^A are defined analogously, using nondeterministic and co-nondeterministic Turing machines, respectively.

For example, $\Sigma_1^{\mathsf{P}} = \mathsf{NP}$, $\Pi_1^{\mathsf{P}} = \mathsf{coNP}$, and $\Delta_2^{\mathsf{P}} \coloneqq \mathsf{P}^{\mathsf{NP}}$ is the class of problems solvable in polynomial time with access to an oracle for an NP-complete problem.

This definition implies that each level of the hierarchy is contained within the next. That is:

- $\Sigma_i^{\mathsf{P}} \subseteq \Delta_{i+1}^{\mathsf{P}} \subseteq \Sigma_{i+1}^{\mathsf{P}}$
- $\Pi_i^{\mathsf{P}} \subseteq \Delta_{i+1}^{\mathsf{P}} \subseteq \Pi_{i+1}^{\mathsf{P}}$

The union of all classes in this hierarchy forms the polynomial-time hierarchy, denoted by PH.

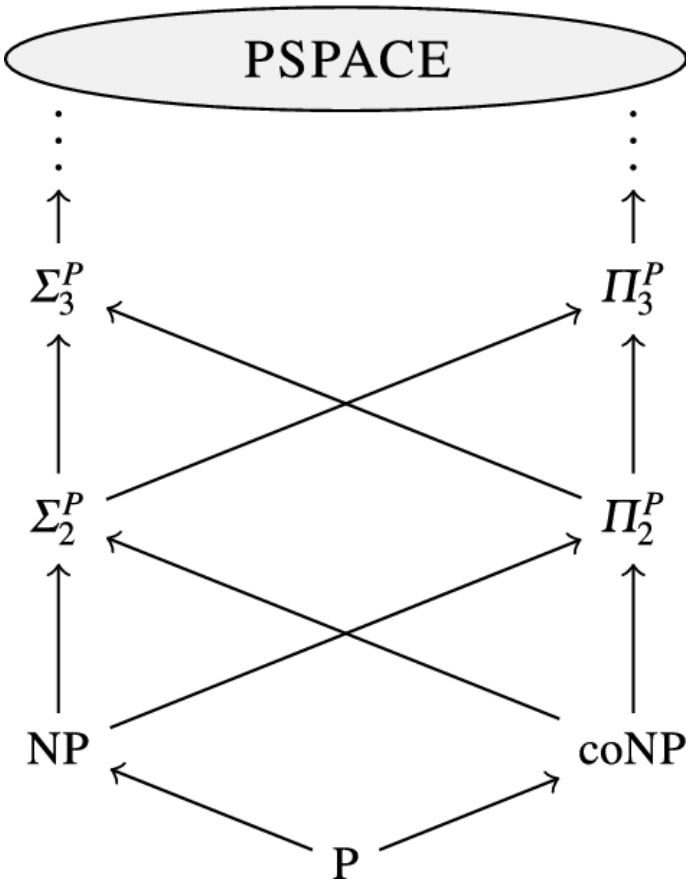


Figure 2.3: Polynomial Time Hierarchy where the arrows denote the inclusion: image from [8]

.

Chapter 3

State of the Art

3.1 The Graph Isomorphism Problem

The Graph Isomorphism (GI) problem is one of the best-known problems in theoretical computer science and graph theory. It has attracted so much attention from algorithm designers that it was referred to as a "disease" by researchers [55]. GI belongs to the complexity class NP, whose certificate is a claimed isomorphism between the two graphs: for every pair of vertices, one simply checks that they are adjacent in one graph if and only if their images under the claimed isomorphism are adjacent in the other. Yet it remains unknown whether it can be solved in polynomial time, that is, whether GI belongs to P. Likewise, GI is not known to be NP-complete, and its relationship with coNP also remains unresolved. As a result, the Graph Isomorphism problem continues to occupy a unique position in computational complexity theory.

During the 1970s, GI became one of the emblematic "open problems" of complexity theory: it was unknown whether it belonged to P, was NP-complete, or occupied an intermediate position between the two [25]. Today, the possibility that GI is NP-complete is considered highly unlikely. Indeed, it has been shown that proving GI NP-complete would imply that the polynomial-time hierarchy (PH) collapses to its second level [26, 57]. Such a collapse is considered extremely unlikely, as complexity theorists widely conjecture that all levels of the polynomial hierarchy are distinct [1]. Consequently, GI is regarded as one of the strongest candidates for an NP-intermediate problem.

Although its exact complexity remains unknown, several important theoretical results have progressively clarified GI's position within the complexity landscape. For instance, Graph Isomorphism has been shown to belong to SPP. Furthermore, Torán proved that GI is hard for a large collection of logarithmic-space complexity classes, highlighting that the problem also occupies a central role in the study of space complexity [62]. These results illustrate that, despite decades of research, the exact complexity of GI remains remarkably subtle.

Progress has also been made from an algorithmic perspective. For more than thirty years, the fastest proven algorithm for Graph Isomorphism had a running time of $e^{\mathcal{O}(\sqrt{n \log n})}$ [7]. A major breakthrough came in 2016, when László Babai introduced the first quasi-polynomial-time algorithm, running in time $e^{\log(n)^{\mathcal{O}(1)}}$ [4]. The algorithm builds upon previous advances in Graph Isomorphism algorithms and handles highly symmetric cases

using group-theoretic (introduced in more details in a few paragraphs) local certificates[1] and combinatorial canonical partitioning techniques. Although this result does not settle whether GI belongs to P, it further supports the belief that the problem is NP-intermediate rather than NP-complete, since an NP-complete problem admitting a quasi-polynomial-time algorithm would imply that every problem in NP also has a quasi-polynomial-time algorithm. This would be a major and unexpected consequence in computational complexity theory, as it would refute the Exponential Time Hypothesis (ETH) [32].

Despite the difficulty of the general Graph Isomorphism problem, efficient algorithms can be obtained for specific classes of graphs. In particular, Babai, Erdős and Selkow showed that in the Erdős–Rényi random graph model $\mathcal{G}(n, 1/2)$, where each edge is independently included with probability 1/2, almost all graphs can be canonically labelled in linear time by a simple refinement procedure [6]. This result illustrates that, although carefully constructed graphs can remain challenging, random graphs are typically easy to distinguish due to their lack of structural symmetries.

Despite these advances, the precise complexity of Graph Isomorphism remains unresolved. Nevertheless, polynomial-time algorithms are known for numerous restricted graph classes. These classes are typically characterised by structural constraints, such as excluded minors[2] (see [53] in Russian, as cited in [45]), or excluded topological subgraphs[3] [27]. Prominent examples include graphs of bounded degree[4] [41], bounded tree-width [10][5], and, more recently, graphs of clique-width[6] at most three [16]. Graphs of bounded genus[7] were also claimed to admit a polynomial-time algorithm [21]; however, the correctness of the original proof was later questioned [49].

For some restricted graph classes, even stronger complexity bounds are known. In particular, Graph Isomorphism for planar graphs[8] and interval graphs[9] belongs to the complexity class LOGSPACE (cf. Definition 15), representing a significant improvement in memory complexity [17, 39].

As increasingly sophisticated techniques have been developed, polynomial-time algorithms have been obtained for a growing number of graph classes. However, many of these algorithms remain primarily of theoretical interest and are difficult to implement efficiently. Notable exceptions include trees [2], planar graphs [30], and interval graphs [13], for which practical algorithms are available.

Conversely, restricting the input to particular graph families does not necessarily simplify the problem. Indeed, Graph Isomorphism remains GI-complete[10] on several seemingly

[1] Efficiently verifiable structures that reveal properties of local graph structures without requiring a complete global analysis.

[2] A graph H that cannot be formed within another graph G through any sequence of edge deletions, vertex deletions or edge contractions.

[3] Same idea of a minor but only allow for the contraction of edges adjacent to degree-two vertices.

[4] A graph whose maximum degree is bounded by a fixed constant d.

[5] An integer measuring, informally, how far the graph is from being a tree. See Reinhard Diestel's book on graph theory for further information [18].

[6] It measures the structural complexity of a graph by the minimum number of labels required to construct it using simple graph operation.

[7] The genus of a graph is the minimum number of handles required on a surface to embed the graph without edge crossings.

[8] Graph that can be drawn on a flat surface so that its edges intersect only at their shared endpoints.

[9] A graph whose vertices correspond to a set of intervals on a line. Two vertices are connected by an edge if and only if their corresponding intervals overlap or share an endpoint.

[10] A specific family or class of graphs has an isomorphism problem that is as hard as the general GI

restricted classes, including bipartite graphs[11], chordal graphs[12], graphs of bounded degeneracy[13], and graphs of bounded expansion[14] [27].

To solve the Graph Isomorphism (GI) problem, three main types of algorithms can be considered [27]: graph-theoretic algorithms, group-theoretic algorithms, and combinatorial algorithms.

The first family consists of *graph-theoretic algorithms*, classical algorithms that exploit structural properties of specific graph classes. A representative example is the algorithm of Hopcroft and Wong [31] for planar graphs, which relies on the decomposition of the graph into triconnected (3-connected) components and uses graph-theoretic techniques to solve the isomorphism problem efficiently.

The second family comprises *group-theoretic algorithms*, based on computing generators for the automorphism group of a graph. These methods leverage group-theoretic properties and have dominated GI research since the early 1980s. Indeed, GI and the problem of computing a generating set for a graph's automorphism group can each be reduced to the other in polynomial time [42]: the two problems are polynomial-time equivalent, so that solving either one suffices to solve the other. This is the case of Luks' algorithm [41], which solves the isomorphism problem on graphs of bounded degree by exploiting the structure of the automorphism group to reduce the problem to a sequence of simpler instances.

Finally, *combinatorial algorithms* are simple, general-purpose procedures that do not rely on the specific structure of graph classes. All combinatorial algorithms are based on the idea of *colour refinement* (the 1-dimensional version of the Weisfeiler-Leman algorithm [35]), which aims to compute a *stable colouring* of a graph, as illustrated in Algorithm 1. Colour refinement is one of the simplest ideas for testing graph isomorphism and has been refined, extended, and incorporated into numerous algorithms over the years. However, the earliest version of the algorithm dates back to 1965, when it was introduced by H. L. Morgan for the study of chemical structures [48].

Algorithm 1 Colour Refinement Algorithm [27]

Input: A graph $G = (V, E)$
Initialisation: Assign the same colour to every vertex.
repeat
    Refinement Step:
    for all vertices v, w with the same colour **do**
        if v and w have different numbers of neighbours in some colour class **then**
            Assign different colours to v and w.
        end if
    end for
until the colouring is stable
Return: the stable colouring.

problem.

[11] A network of vertices divided into two disjoint sets, where every edge connects a vertex in one set to a vertex in the other.

[12] An undirected graph where every cycle of length 4 or greater has a chord.

[13] A graph has bounded degeneracy d if every subgraph contains a vertex with a degree of d or less.

[14] Graphs of bounded expansion are families of sparse graphs (a graph whose number of edges is small compared to its number of vertices) where all shallow minors (a minor obtained by restricting the depth of the contractions) have bounded average density.

To use colour refinement as an isomorphism test, we apply it to the disjoint union of the input graphs G and G'. The algorithm *distinguishes* G and G' if there exists a colour c in the stable colouring such that G and G' contain different numbers of vertices of colour c. Colour refinement *identifies* a graph G if it distinguishes G from all non-isomorphic graphs; that is, no non-isomorphic graph shares the same stable colouring as G. However, if two graphs share the same stable partition, this is not a sufficient condition to prove that they are isomorphic; it is only a necessary one. Therefore, colour refinement does not fully solve the Graph Isomorphism problem, but rather provides a useful indication.

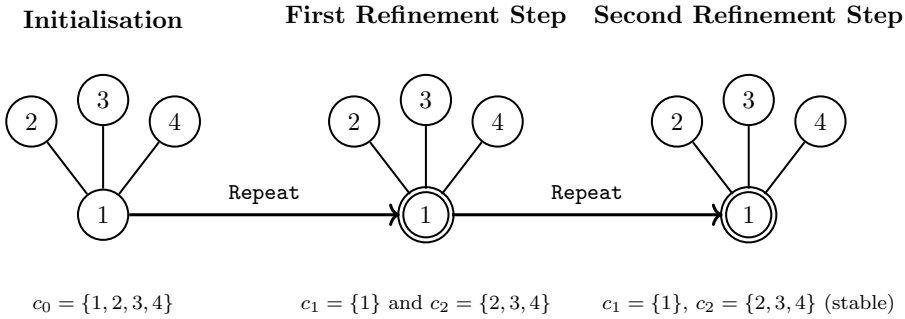


Figure 3.1: Example of colour refinement on $K_{1,3}$.

Figure 3.1 illustrates the execution of colour refinement on $K_{1,3}$. Initially, all four vertices belong to the same colour class c_0. During the first refinement step, the algorithm compares the neighbourhood profiles of the vertices. Vertex 1 has three neighbours, all of colour c_0, whereas vertices 2, 3, and 4 each have exactly one neighbour of colour c_0. Consequently, the refinement produces two colour classes: vertex 1 receives one colour, while vertices 2, 3, and 4, which remain indistinguishable from one another, receive another. A second refinement step leaves the colouring unchanged. Vertex 1 is already isolated in its own colour class, while vertices 2, 3, and 4 still each have exactly one neighbour of the first colour and therefore remain indistinguishable. The resulting colouring is stable, and the algorithm terminates.

Although originally developed in the context of GI, colour refinement has since found applications well beyond it: it has been used to reduce the cost of solving linear programs [28] and serves as the foundation of the Weisfeiler-Lehman graph kernels used in machine learning [59].

Regarding the time complexity, for a graph G with n vertices and m edges, a stable colouring can be reached after at most n refinement steps. In fact, it can be computed in time $\mathcal{O}((n+m)\log n)$ [12], and has a lower bound of $\Omega((n+m)\log n)$ for the natural class of iterative partition refinement algorithms [9]. This gives a tight bound of $\Theta((n+m)\log n)$ for such algorithms.

Now that we have introduced colour refinement, we can turn to the types of combinatorial algorithms that build upon it. The most direct generalisation is the *Weisfeiler-Leman algorithm* (or k-WL) [35]. Its underlying principle remains largely the same as that of colour refinement, except that instead of comparing individual vertices, the algorithm operates on k-tuples of vertices. The central concept it introduces is that of *i-neighbours*: for $\vec{v}, \vec{w} \in V^k$ and $i \in \{1, \ldots, k\}$, the tuples $\vec{v}$ and $\vec{w}$ are said to be i-neighbours if $v_j = w_j$ for all $j \neq i$.

Algorithm 2 k-dimensional Weisfeiler-Leman (k-WL) algorithm

Require: Graph $G = (V, E)$, integer $k \geq 1$
Ensure: Stable colouring of V^k

```
Initialization:
for all tuples v⃗, w⃗ ∈ V^k do
    if v⃗ ≇ w⃗ then                ▷ No isomorphism between induced subgraphs
        Assign different initial colours to v⃗ and w⃗
    end if
end for
repeat
    Refinement step:
    for all tuples v⃗, w⃗ ∈ V^k do
        for i = 1 to k do
            Compare multisets of colours of i-neighbours of v⃗ and w⃗
            if they differ then
                Assign different colours to v⃗ and w⃗
            end if
        end for
    end for
until the colouring is stable
```

An interesting result is that for all graphs G and H, 2-WL can distinguish G and H if and only if colour refinement can distinguish them. In other words, despite the fact that 2-WL operates on pairs of vertices and provides a richer representation of the graph structure, it has the same distinguishing power as colour refinement for the graph isomorphism problem [27].

However, the situation changes when considering dimensions higher than two. For every $k \geq 3$, k-WL is strictly more powerful than $(k-1)$-WL. In particular, while 2-WL and colour refinement have the same distinguishing power, 3-WL can distinguish graph pairs that are indistinguishable by 1-WL and 2-WL.

The reason behind this improvement is that k-WL considers tuples of k vertices instead of individual vertices. Consequently, it can capture more global structural relationships between vertices. In contrast, 1-WL has a more local perspective, as it only refines vertex colours based on the multiset of colours of their neighbours. By increasing k, the algorithm is able to analyse interactions between several vertices simultaneously and therefore detect structural differences that remain invisible to lower-dimensional versions.

For example, 3-WL can solve the graph isomorphism problem for planar graphs [36]. This result illustrates the increasing expressive power of higher-dimensional Weisfeiler-Leman algorithms.

Regarding its complexity, for a graph G with n vertices, the stable colouring of k-WL can be reached after at most n^k refinement rounds. The stable colouring can then be computed in time $\mathcal{O}(n^k \log n)$ [27].

Beyond this tuple-based generalisation, applying higher-dimensional Weisfeiler-Leman algorithms quickly becomes computationally expensive in practice. Consequently, most modern graph isomorphism software rely on the one-dimensional version, namely *colour refinement*, introduced in the previous section. For the vast majority of graphs, colour

refinement already provides a sufficiently informative partition of the vertex set.

When colour refinement alone fails to distinguish two graphs, modern isomorphism algorithms typically resort to a second family of techniques known as *individualisation-refinement*. This approach combines the individualisation of carefully chosen vertices with repeated applications of colour refinement. The technique was first introduced by Parris and Read [51], as cited in [45]. Individualising a vertex breaks graph symmetries, allowing colour refinement to be applied again and often producing a significantly finer partition of the vertices [29].

The first practical implementation of this paradigm appeared in *Nauty*, whose central idea is to construct a *search tree* in which individualisation and refinement are applied recursively. During the exploration of this search tree, a *node invariant* is computed at every node to guide the search and prune unnecessary branches. Once the traversal is complete, the information gathered throughout the search is used to compute a *canonical labelling* of the graph, that is, a unique representation (or canonical form) of the graph. This canonical labelling lies at the heart of solving the Graph Isomorphism problem, since two graphs are isomorphic if and only if their canonical forms are identical. These notions will be revisited in much greater detail in the next chapter.

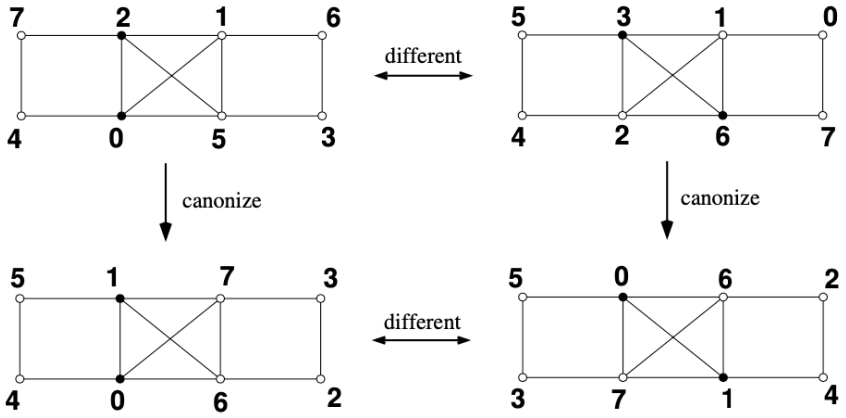


Figure 3.2: Example of GI with canonical labelling from [47]

Over time, several other GI solvers based on the individualisation-refinement paradigm have emerged, each building on or extending the ideas underlying *Nauty*. *Saucy*, introduced in 2004, follows the same principle but was designed to outperform *Nauty* on sparse graphs [15]. A few years later, *Bliss* was released; it also computes canonical forms but incorporates additional techniques that improve performance on more complex graphs ([34], as cited in [45]).*Conauto*, introduced in 2009, has shown remarkable efficiency on random graphs and on certain families of hard instances [40]. Finally, *Traces* was developed [52] as a tool able to compete with *Nauty* on the Graph Isomorphism problem; today, Traces and *Nauty* are tightly integrated and distributed together [45].

Unfortunately, despite their remarkable practical performance, all the individualisation-refinement algorithms cannot achieve polynomial worst-case running time. In 2017, David Neuen and Pascal Schweitzer proved that every algorithm belonging to this family has an exponential lower bound [50]. In other words, there exist graph families on which every individualisation-refinement algorithm necessarily requires exponential time. Nevertheless,

these algorithms remain the most successful practical approach for solving the Graph Isomorphism problem. This striking contrast illustrates how far practical algorithms still are from the best theoretical results, such as Babai's quasi-polynomial-time algorithm.

Finally, a rather different approach to the Graph Isomorphism problem is based on linear programming. Let A and B denote the adjacency matrices of two graphs G and H. Then, G and H are isomorphic if and only if there exists a permutation matrix X such that

$$X^\top AX = B.$$

This formulation can be relaxed using LP techniques, notably the Sherali-Adams hierarchy. In particular, level-k Sherali-Adams relaxations offer a powerful framework that has been investigated for graph isomorphism testing [27].

Chapter 4

Nauty Algorithm

In this section, we will present McKay's general algorithm, *Nauty*, in detail. This section is therefore largely inspired by his paper, *Practical Graph Isomorphism* [45], with the aim of providing a clearer and more accessible presentation.

4.1 Introduction of the algorithm

Nauty relies on the notion of a canonical form to achieve its main objective: finding a canonical labelling of a graph and, consequently, determining its automorphism group.

The canonical form allows us to bypass the intrinsic difficulty of the graph isomorphism problem. Instead of directly comparing two graphs, we associate each graph with a unique and standardised representative, called its canonical graph (or canonical form). Two graphs are then isomorphic if and only if their canonical forms are identical. In this way, an equivalence problem is transformed into an equality problem, which is significantly easier to handle computationally. For instance, when comparing a large collection of graphs, it is more efficient to compute the canonical form of each graph once and then compare these canonical forms, rather than perform an isomorphism test for every pair of graphs.

We begin by introducing the algorithm itself and describing the different functions that compose it, as well as the process used to compute the canonical form of a graph. We then explain how the automorphism group can be constructed from this canonical form.

4.2 Search tree construction

First of all, the algorithm itself is built upon a very specific type of graph: a tree. More specifically, it relies on a search tree.

Informally, a search tree is a tree structure in which each node represents a state, and each branch corresponds to a specific action leading to another state. Since different actions produce different branches, the tree allows us to systematically explore all possible states starting from an initial state, with the goal of reaching a desired final state. In other words, we are performing a search over a tree structure, which explains the term *search tree*.

However, its particularity is that the nodes of this tree correspond to sequences of vertices of a coloured graph. As we move down the tree, these sequences grow longer. These sequences define a colouring of the graph obtained by first assigning unique colours to the vertices in the sequence and then deducing, in a "controlled" manner, the colouring of the remaining vertices. The leaves of this tree correspond to sequences whose derived colouring is discrete. The root of this tree is therefore the empty sequence.

To correctly construct this tree and find our canonical form, we will need several functions:

- *A refinement function*: Starting from a partition, this function allows us to construct a new partition that is finer than the previous one.
- *A target cell selection function*: Starting from a partition, this function allows us to choose a cell from this partition in order to create the "children" of the previous partition.
- *A node invariant function*: A function that allows us to efficiently compare the different nodes/leaves of our search tree in order to determine the canonical form of the graph.

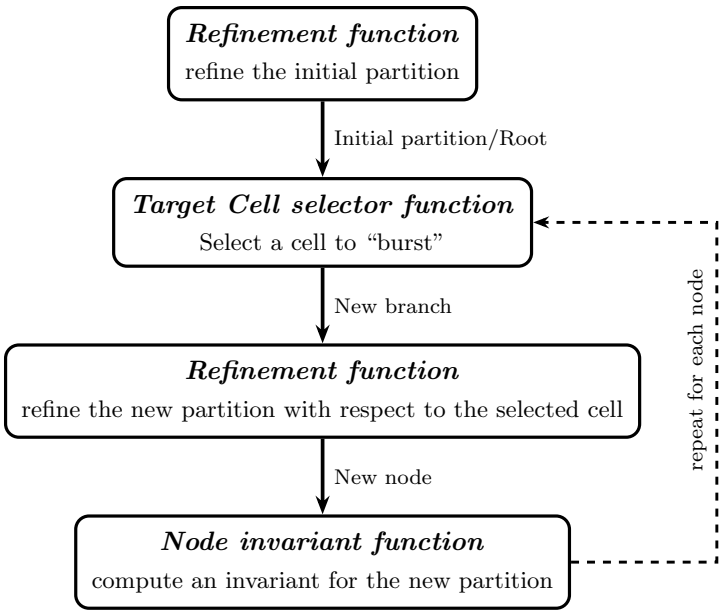


Figure 4.1: *search tree* construction in *Nauty*

These three functions are the fundamental components required to construct our search tree and find our canonical form. Before defining them formally, we will informally explain how they work and how they interact with each other to build our search tree.

Starting from a graph, we first apply the refinement function to obtain an equitable partition (we will come back to this later when discussing the implementation of this function). This partition constitutes the root of our search tree.

Then, as long as we do not have a discrete partition, we apply the target cell selector to choose a cell from the current partition. Each branch represents a choice of element from the parent node's selected cell. This cell is then split into two cells using an individualisation method (which will also be detailed later). For each branch, we apply the refinement function again to obtain a new partition, thereby creating a new node. Finally,

on each node, we apply the node invariant function, which will be useful for determining the canonical form of our graph. We repeat this process until we obtain discrete partitions, which correspond to the leaves of our search tree. In summary, each node represents a partition, and each branch corresponds to the individualisation of a vertex from a chosen cell of the parent node.

However, there exist additional functions that are more specific to the implementation of the algorithm and that will be explained in the implementation section of this document.

4.2.1 Formal definitions of the search tree functions

Now that we have introduced the different functions composing our algorithm, we can formally define them and establish the properties of the associated search tree.

Definition 17. A *refinement function* is a function

$$R : \mathcal{G} \times \Pi \times V^* \to \Pi$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$:

R1. $R(G, \pi, \sigma) \preccurlyeq \pi$.

R2. if $v \in \sigma$[1], then $\{v\}$ is a cell of $R(G, \pi, \sigma)$.

R3. for any $g \in S_n$, we have $R(G^g, \pi^g, \sigma^g) = R(G, \pi, \sigma)^g$.

In other words, the refinement function returns a colouring that is finer than π, and any vertex belonging to the sequence σ must form a singleton cell in the refined colouring. Furthermore, the function is equivariant under permutation: applying any $g \in S_n$ to the graph, the colouring, and the vertex sequence before or after the refinement yields the same result.

Definition 18. The *target cell selector function* is a function

$$T : \mathcal{G} \times \Pi \times V^* \to 2^V$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$:

T1. if $R(G, \pi, \sigma)$ is discrete, then $T(G, \pi, \sigma) = \emptyset$.

T2. if $R(G, \pi, \sigma)$ is not discrete, then $T(G, \pi, \sigma)$ is a non-singleton cell of $R(G, \pi, \sigma)$.

T3. for any $g \in S_n$, we have $T(G^g, \pi^g, \sigma^g) = T(G, \pi, \sigma)^g$.

In other words, when the refinement function returns a discrete partition (corresponding to a leaf of the search tree) the target cell selector returns $\emptyset$. When the partition is non-discrete (corresponding to an internal node) it returns a non-singleton cell of that partition. As with the refinement function, the target cell selector is equivariant under permutation: applying any $g \in S_n$ to the inputs before or after the selection yields the same result.

Now that all the formal definitions of the tree have been established, we can define the search tree $\mathcal{T}(G, \pi_0)$, which depends on an initial coloured graph (G, π_0). All the nodes of the tree are elements of V^*. This search tree has some characteristics to keep in mind:

[1] Here, the notation $v \in \sigma$ is used informally to indicate that the vertex v appears in the sequence σ.

- The root of $\mathcal{T}(G, \pi_0)$ is the empty sequence ().
- if σ is a node of $\mathcal{T}(G, \pi_0)$, let W $= T(G, \pi_0, \sigma)$. Then the children of π are $\{\sigma \mid\mid w : w \in W\}$.
- a node σ of $\mathcal{T}(G, \pi_0)$ is a leaf iff $R(G, \pi_0, \sigma)$ is discrete.
- for any node σ of $\mathcal{T}(G, \pi_0)$, define $\mathcal{T}(G, \pi_0, \sigma)$ to be the subtree of $\mathcal{T}(G, \pi_0)$ consisting of σ and all its descendants.

To demonstrate the correctness of the algorithm, the pruning procedure that we will introduce, and to prove that we indeed obtain the canonical form of the graph, we need to establish several properties of this search tree.

Lemma 1. *For any $G \in \mathcal{G}, \pi_0 \in \Pi, g \in S_n$ we have $\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$.*

Proof. Let $\sigma = (\sigma_1, \ldots, \sigma_k)$ be a node of $\mathcal{T}(G, \pi_0)$, and let $\sigma^g = (\sigma_1^g, \ldots, \sigma_k^g)$ $\mathcal{T}(G^g, \pi_0^g)$. We first show that $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$.

We proceed by induction on s, for $0 \leq s \leq k$, showing that $[\sigma^g]_s = (\sigma_1^g, \ldots, \sigma_s^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

Base case ($s = 0$): The empty sequence () is the root of any search tree. So $[\sigma^g]_0$ is trivially a node of $\mathcal{T}(G^g, \pi_0^g)$.

Inductive step: Assume $[\sigma^g]_s = (\sigma_1^g, \ldots, \sigma_s^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

Since $(\sigma_1, \ldots, \sigma_s, \sigma_{s+1})$ is a node of $\mathcal{T}(G, \pi_0)$, the element σ_{s+1} was individualised to go from level s to level $s+1$. Since we apply g to the entire structure, σ_{s+1} is also mapped to σ_{s+1}^g, which is equivalent to directly individualising σ_{s+1}^g in G^g with π_0^g. Hence $[\sigma^g]_{s+1} = (\sigma_1^g, \ldots, \sigma_s^g, \sigma_{s+1}^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

By induction, σ^g is a node of $\mathcal{T}(G^g, \pi_0^g)$, hence $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$.

The reverse inclusion follows by applying the same argument with g^{-1} in place of g, which gives $\mathcal{T}(G^g, \pi_0^g)^{g^{-1}} \subseteq \mathcal{T}(G, \pi_0)$, and therefore $\mathcal{T}(G^g, \pi_0^g) \subseteq \mathcal{T}(G, \pi_0)^g$. □

Corollary 2. *Let σ be a node of $\mathcal{T}(G, \pi_0)$ and let $g \in Aut(G, \pi_0)$. Then σ^g, is a node of $\mathcal{T}(G, \pi_0)$ and $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$.*

Proof.
Part 1 (σ^g is a node of $\mathcal{T}(G, \pi_0)$):
By Lemma 1, we have:

$$\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$$

Since $g \in \mathrm{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and therefore:

$$\mathcal{T}(G, \pi_0) = \mathcal{T}(G, \pi_0)^g$$

Since σ is a node of $\mathcal{T}(G, \pi_0)$, σ^g is a node of $\mathcal{T}(G, \pi_0)^g = \mathcal{T}(G, \pi_0)$.

Part 2 ($\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$):
We apply Lemma 1 to the subtree rooted at σ. Since g maps each node of the subtree rooted at σ to a node of the subtree rooted at σ^g, and since $g \in \mathrm{Aut}(G, \pi_0)$ implies $G^g = G$ and $\pi_0^g = \pi_0$, the same argument as in Lemma 1 gives both inclusions:

$$\mathcal{T}(G, \pi_0, \sigma)^g \subseteq \mathcal{T}(G, \pi_0, \sigma^g) \quad \text{and} \quad \mathcal{T}(G, \pi_0, \sigma^g) \subseteq \mathcal{T}(G, \pi_0, \sigma)^g$$

and therefore $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$. □

Lemma 3. *Let σ be a node of (G, π_0) and let $\pi = R(G, \pi_0, \sigma)$. Then $\mathrm{Aut}(G, \pi)$ is the point-wise stabiliser of σ in $\mathrm{Aut}(G, \pi_0)$.*

Proof. We show both inclusions to establish the equality.

Part 1 ($\mathrm{Aut}(G, \pi) \subseteq$ point-wise stabiliser of σ):
Let $g \in \mathrm{Aut}(G, \pi)$. By R2, for each $v \in \sigma$, $\{v\}$ is a singleton cell of $\pi = R(G, \pi_0, \sigma)$. Since g is an automorphism of (G, π), it must map each cell of π to a cell of π. A singleton $\{v\}$ can only be mapped to itself, hence $v^g = v$ for every $v \in \sigma$, so g stabilises σ point-wise.

Part 2 (point-wise stabiliser of $\sigma \subseteq \mathrm{Aut}(G, \pi)$):
Let $g \in \mathrm{Aut}(G, \pi_0)$ be such that g stabilises σ point-wise, that is $\sigma^g = \sigma$. We want to show that $g \in \mathrm{Aut}(G, \pi)$ (so, $\pi^g = \pi$). By R3 applied to G, π_0 and σ:

$$R(G^g, \pi_0^g, \sigma^g) = R(G, \pi_0, \sigma)^g$$

Since $g \in \mathrm{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and since g stabilises σ point-wise, $\sigma^g = \sigma$. Therefore:

$$R(G, \pi_0, \sigma) = R(G, \pi_0, \sigma)^g$$

which gives exactly $\pi = \pi^g$, and hence $g \in \mathrm{Aut}(G, \pi)$. □

Now that we have properly defined our search tree, we can discuss the node invariant function and how it allows us to compute the canonical form of the graph.

Definition 19. Let Ω be some totally ordered set. A *Node invariant* is a function

$$\phi : \mathcal{G} \times \Pi \times V^* \to \Omega$$

Such that, for any $\pi_0 \in \Pi$, $G \in \mathcal{G}$ and distinct $\sigma, \sigma' \in \mathcal{T}(G, \pi_0)$:

ϕ1. if $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$, then for every leaf $\sigma_1 \in \mathcal{T}(G, \pi_0, \sigma)$ and leaf $\sigma_1' \in \mathcal{T}(G, \pi_0, \sigma')$ we have $\phi(G, \pi_0, \sigma_1) < \phi(G, \pi_0, \sigma_2)$.

ϕ2. if $\pi = R(G, \pi_0, \sigma)$ and $\pi' = R(G, \pi_0, \sigma')$ are discrete, then $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma') \Leftrightarrow G^\pi = G^{\pi'}$.

ϕ3. for any $g \in S_n$, we have $\phi(G^g, \pi_0^g, \sigma^g) = \phi(G, \pi_0, \sigma)$.

Property 1 ensures that the ordering induced by ϕ is consistent across the search tree: if two sequences of equal length satisfy $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$, then this strict inequality propagates to all descendant leaves, so no leaf below σ can exceed any leaf below σ'. Property 2 states that at the leaves (where both refined partitions are discrete) the invariant captures the canonical labelling exactly: two leaves yield equal invariants if and only if they produce the same canonically labelled graph. Property 3 expresses that ϕ is invariant under permutation: applying any $g \in S_n$ to the graph, the colouring, and the vertex sequence before computing ϕ leaves the result unchanged.

We can say that leaves σ, σ' are *equivalent* if $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma')$. If this is the case, there is a unique $g \in \mathrm{Aut}(G, \pi_0)$ such that $\sigma^g = \sigma'$, with $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$. In other words, it connects equivalence of leaves to the automorphism group. We recall that $R(G, \pi_0, \sigma)$ is the discrete partition obtained by applying the refinement function along the sequence σ, and can therefore be viewed as a labelling of the vertices. Since both leaves are equivalent, their node invariants are equal, which means there exists a permutation g such that:

$$R(G, \pi_0, \sigma)^g = R(G, \pi_0, \sigma')$$

Isolating g, we obtain:

$$g = R(G, \pi_0, \sigma)^{-1} \cdot R(G, \pi_0, \sigma')$$

If this permutation turns out to be an automorphism of (G, π_0), it means that both leaves describe the same graph up to a relabelling of vertices, and are therefore equivalent.

By Corollary 2, if σ is a leaf of $\mathcal{T}(G, \pi_0)$, then all leaves of the form σ^g, where $g \in \mathrm{Aut}(G, \pi_0)$, are also leaves of $\mathcal{T}(G, \pi_0)$, and, due to the properties of ϕ, they all share the same ϕ-value, while no other leaf has this value. Consequently, for each leaf σ, we can say that:

$$\mathrm{Aut}(G, \pi_0) = \big\{R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1} : \sigma' \text{ is a leaf of } \mathcal{T}(G, \pi_0) \text{ with } \phi(G, \pi_0, \sigma') = \phi(G, \pi_0, \sigma)\big\}.$$

This means that the automorphism group is generated entirely from leaves sharing the same ϕ-value as a fixed reference leaf σ: for each such leaf σ', composing the permutation $R(G, \pi_0, \sigma')$ with the inverse of $R(G, \pi_0, \sigma)$ yields an automorphism of (G, π_0), and every automorphism arises this way. This holds for any choice of reference leaf σ.

We first define the *canonical invariant* of (G, π_0):

$$\phi^*(G, \pi_0) \coloneqq \max\{\phi(G, \pi_0, \sigma) : \sigma \text{ is a leaf of } \mathcal{T}(G, \pi_0)\}.$$

Let σ^* be any leaf of $\mathcal{T}(G, \pi_0)$ achieving this maximum. We can now define the canonical form of (G, π_0) as

$$C(G, \pi_0) \coloneqq (G, \pi_0)^{R(G, \pi_0, \sigma^*)}.$$

By the properties of ϕ, $C(G, \pi_0)$ thus defined is independent of the choice of σ^*; in particular, any leaf achieving the maximum may be picked arbitrarily, and the resulting canonical form will always be the same.

In other words, the canonical invariant represents the maximum ϕ-value achieved among all leaves of the search tree, and σ^* is a leaf that achieves this maximum. The canonical form $C(G, \pi_0)$ is then obtained by applying the permutation associated with the leaf σ^*, namely $R(G, \pi_0, \sigma^*)$ (our canonical labelling), to the initial graph (G, π_0). Here, we allow ourselves to use the refinement function itself to denote the permutation associated with the leaf σ^*. Indeed, since this leaf is discrete, it corresponds to an ordered partition of $V(G)$ into singletons (an ordering of the vertices); as the vertex set is already identified with $\{1, \ldots, n\}$, this ordering genuinely defines a permutation, and thus an element of S_n.

4.2.2 Implementation strategies

Now that we have all the theoretical tools in hand, we can begin discussing the implementation of the algorithm. Our implementation is based on the definitions of the algorithm, but it is not a direct or simplistic reproduction of its source code.

Implementation of the refinement function

In order to ensure that the properties of the refinement function defined in the search tree are respected, we must implement an algorithm that outputs a colouring that is equal to (or finer than) the original colouring. This motivates the need for an algorithm that

computes a coarsest equitable colouring, in order to prevent refinement from being too aggressive and skipping intermediate structure in the search tree.

To construct our refinement function, we need two auxiliary functions: a function that refines a colouring so that it becomes an equitable colouring, and an individualisation function.

Definition 20. The function $F(G, \pi, \alpha)$ is our refinement algorithm (implemented in a label-invariant manner).

pseudo-code of $F(G, \pi, \alpha)$ (from [45])

```
1: Data: π is the input colouring and α is a sequence of some cells of π
2: Result: the final value of π is the output colouring (equitable)
3: while α is not empty and π is not discrete do
4:     Remove some element W from α
5:     for each cell X of π do
6:         Let X_1, ..., X_k be the fragments of X distinguished according to the number
           of edges from each vertex to W
7:         Replace X by X_1, ..., X_k in π
8:         if X ∈ α then
9:             Replace X by X_1, ..., X_k in α
10:        else
11:            Add all but one of the largest of X_1, ..., X_k to α
12:        end if
13:    end for
14: end while
```

This function is, at first glance, relatively straightforward to understand. However, it is important to carefully examine the different steps involved in the refinement process. We therefore describe these steps in more detail below.

As stated in the pseudo-algorithm, we are given a graph G, a colouring π, and a sequence of cells of π that will be used to refine the colouring. As long as not all cells in the sequence have been processed and the colouring is not yet discrete, we continue refining the colouring.

At each iteration, we remove a cell W from the sequence. Then, for every cell X of the current colouring, we split X into several cells $X_1, \ldots, X_k$ according to the number of edges each vertex of X has with the vertices of W (with respect to the graph G). We then replace the cell X in the colouring by the newly created cells $X_1, \ldots, X_k$.

If the cell X already belongs to the sequence, we replace it in the sequence by the cells $X_1, \ldots, X_k$. Otherwise, we add all cells among $X_1, \ldots, X_k$ to the sequence except one, namely one of the largest cells. If several cells have the same maximum size, one of them is chosen arbitrarily and is not added to the sequence.

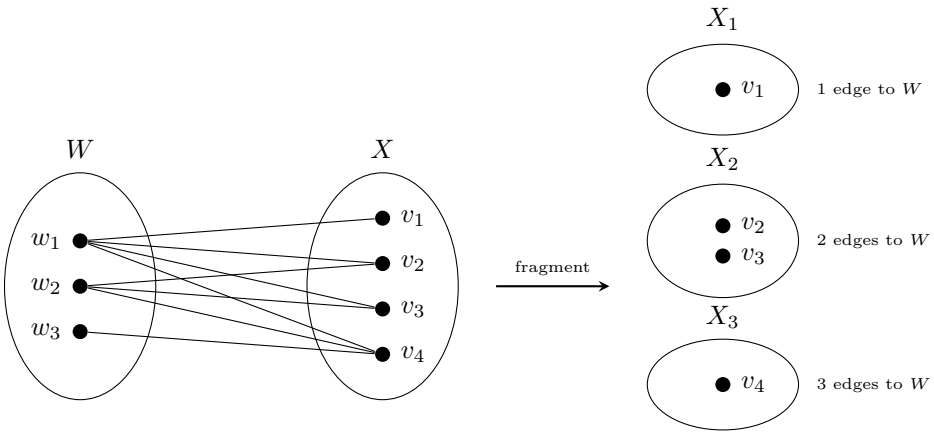


Figure 4.2: Fragmentation of cell X with respect to W

Definition 21 (Individualisation)**.** Let $\pi \in \Pi$ and let $v \in V$ be a vertex belonging to a non-singleton cell of π. The *individualisation function*

$$I : \Pi \times V \to \Pi$$

maps (π, v) to the partition $\pi' = I(\pi, v)$ defined, for every $w \in V$, by

$$\pi'(w) = \begin{cases} \pi(w), & \text{if } \pi(w) < \pi(v) \text{ or } w = v, \\ \pi(w) + 1, & \text{otherwise.} \end{cases}$$

The vertex v is said to be *individualised* in π.

Intuitively, individualisation splits the cell containing v into two parts: the vertex v itself becomes a singleton cell retaining the original position $\pi(v)$, while the remaining vertices of that cell are shifted to position $\pi(v)+1$, immediately after v. Every cell that originally preceded v's cell is left unchanged, and every cell that came after is shifted up by one position to make room; the relative order of the cells is thus preserved. Equivalently, v is assigned a unique colour by removing it from its original cell and placing it into a new singleton cell, inserted immediately before the remaining vertices of that cell.

Now we can define our refinement function. For a sequence of vertices $v_1, v_2, \ldots$, define

- $R(G, \pi_0, ()) = F(G, \pi_0,$ a list of all the cells of $\pi_0)$
- $R(G, \pi_0, (v_1)) = F(G,\ I(R(G, \pi_0, ()), v_1),\ (\{v_1\}))$
- $R(G, \pi_0, (v_1, v_2)) = F(G,\ I(R(G, \pi_0, (v_1)), v_2),\ (\{v_2\}))$
- $R(G, \pi_0, (v_1, v_2, v_3)) = F(G,\ I(R(G, \pi_0, (v_1, v_2)), v_3),\ (\{v_3\}))$
- and so on, whose general form is, for $k \geq 1$:

$$R(G, \pi_0, (v_1, \ldots, v_{k-1}) \mid\mid v_k) = F\Big(G,\ I\big(R(G, \pi_0, (v_1, \ldots, v_{k-1})),\ v_k\big),\ (\{v_k\})\Big).$$

Proposition 4. *The refinement function R defined above satisfies properties (R1) – (R3).*

Proof. We prove each property separately.

Property (R1): By definition, the refinement function R is obtained by repeatedly applying the individualization operator I, with the refinement procedure F applied on top

of the partition produced by I. The operator I only splits an existing cell into two cells, one of which is the singleton containing the individualised vertex, provided that the partition is not already discrete. Therefore, I can only refine the current partition. The refinement function F is designed to produce an equitable partition from the current partition. It can either split existing cells or leave them unchanged, but it never merges cells. Consequently, for every $G \in \mathcal{G}$, $\pi_0 \in \Pi$, and $\sigma \in V^*$, we have

$$R(G, \pi_0, \sigma) \preccurlyeq \pi_0,$$

and property (R1) holds.

Property (R2): In the base case, no vertex is individualised, so the property is trivially satisfied for the empty sequence. For the subsequent steps, the operator I creates a singleton cell containing the individualised vertex. Since F only refines the partition and never merges cells, this singleton remains unchanged after the refinement step. Therefore, every individualised vertex belongs to a singleton cell in the final partition, proving property (R2).

Property (R3): By definition of the individualization operator, for any $g \in S_n$, we have

$$I(\pi^g, v^g) = I(\pi, v)^g.$$

Indeed, the vertex v belongs to the same cell in the transformed partition whether the permutation is applied before or after the individualization step.

Similarly, for the refinement function F, we have

$$F(G^g, \pi^g, \alpha^g) = F(G, \pi, \alpha)^g.$$

The refinement procedure depends only on the structure of the graph and on the colouring of the vertices. In particular, the splitting of cells is determined by adjacency relations rather than by the labels of the vertices. Hence, applying g before or after the refinement leads to equivalent partitions, which proves property (R3).

Therefore, the refinement function R satisfies properties $(R1) - (R3)$. □

In conclusion, the algorithm satisfies the required properties of a refinement function (as shown in Proposition 4) and performs well in practice. However, the refinement step is the most computationally expensive part of the algorithm, which is why special care must be taken in its implementation. Therefore, to reduce the overall running time, we will employ pruning techniques, which will be discussed later.

Implementation of the target cell selector

For the implementation of the target cell selector function in the construction of the search tree, several strategies can be adopted. Choosing the right target cell at each step is important because it influences the shape and depth of the tree, and therefore the overall performance of the search.

In the original version of *Nauty*, McKay recommended selecting the first smallest non-singleton cell, as it increases the chance that the selected cell is part of an orbit of the group that stabilises the current sequence [44]. However, later studies showed that in certain cases, it is better to simply select the first non-singleton cell, regardless of its size [37].

As a result, *Nauty* now supports two strategies for target cell selection that we can choose:

1. Always select the first smallest non-singleton cell.
2. Select the first cell that is joined in a non-trivial way to the largest number of other cells, where a non-trivial join means the number of edges between two cells is strictly between 0 and the maximum possible.

In our implementation we will only use the first strategy.

pseudo-code of targetCellSelector

1: **Data:** a partition $\pi = (X_1, \ldots, X_k)$
2: **Result:** the first non-singleton cell of minimum size in π
3: **return** the first $X \in \pi$ such that $|X| > 1$ and $|X| = \min_{Y \in \pi,\ |Y|>1} |Y|$

Intermediate search tree construction

All the functions required to define the search tree have now been implemented. The following example illustrates the construction of the search tree for the graph $K_{1,3}$ using target-cell selection and equitable refinement.

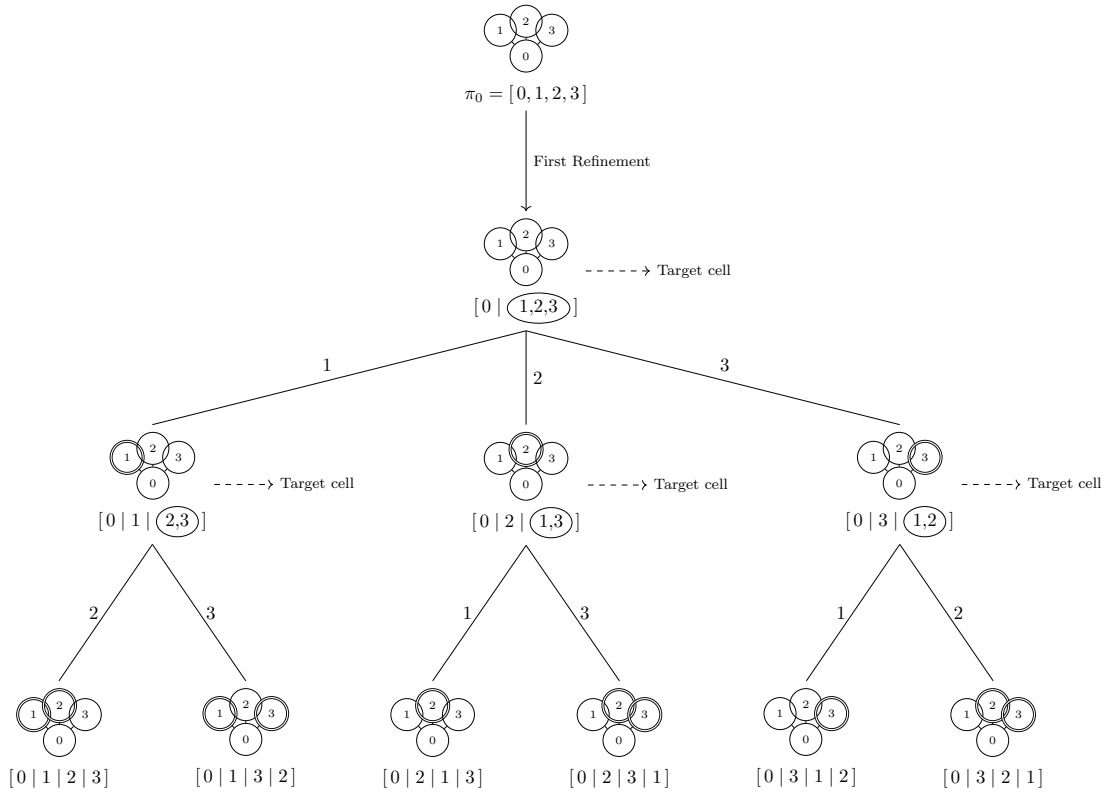


Figure 4.3: Individualised vertices are represented by double circles. The encircled set in each partition denotes the current target cell.

Implementation of the node invariant

Once the search tree construction is in place, the next step is to define the node invariant function, which will allow us to determine the canonical form. To compute this function,

we can rely on two related sources:

1. For each node σ, there is a colouring $R(G, \pi_0, \sigma)$. We can use its properties to extract the number and size of the cells, as well as combinatorial properties of the coloured graph.
2. We can use the intermediate states of the colouring computation from the parent node (in the refinement algorithm), such as the position and size of the cells produced during the refinement procedure, and various edge counts determined during the computation. If $f(\sigma)$ is a function of this information—computed during the calculation of $R(G, \pi_0, \sigma)$ and from the resulting coloured graph—then the vector $(f([\sigma]_0), f([\sigma]_1), \ldots, f(\sigma))$, with lexicographic ordering, satisfies the characteristics required for a node invariant. In *Nauty*, the second source is used: the value of $f(\sigma)$ is an integer, and the pruning rules are applied as each node is computed.

Another potential method is based on the concept of the *quotient graph*[2] $Q(G, \pi)$, assuming that the colouring π of graph G is equitable. All vertices of $Q(G, \pi)$ are cells of π, and they include information about the size and number of the cell. For any two cells $C_1, C_2 \in \pi$, possibly equal, the corresponding vertices in $Q(G, \pi)$ are connected by an edge labelled with the number of edges in G between C_1 and C_2. The node invariant already computed by *Nauty* is a deterministic function of the sequence of quotient graphs $Q(G, R(G, \pi_0, [\sigma]_i))$ for $i = 0, \ldots, |\sigma|$. Therefore, it would be possible to use the sequence of quotient graphs to obtain information about the cells, but this would be far too costly in both time and memory.

In our implementation, we will use the first source described above. We rely on the number and size of the cells, and we apply combinatorial properties of the coloured graph to compute our node invariant. Several invariant methods have been implemented in order to test their efficiency and computational speed.

Since this function is called at each node creation, it is important that its computation remains fast in order to avoid slowing down the algorithm. The following sections detail each of these methods, along with a brief discussion of their computational complexity and how they compare to one another.

Here is the exhaustive list of invariant methods that we have implemented so far:

1. *number of triangles per cell*: for each cell of the partition, we compute the number of triangles contained in that cell. This yields a vector whose i-th entry is the number of triangles contained in the i-th cell of the partition. This vector constitutes the node invariant. During the search, node invariants are then compared lexicographically to determine the best invariant encountered so far.

[2] Informally, the *quotient graph* $Q(G, \pi)$ of a coloured graph (G, π) is the graph whose vertices are the cells of π, with two cells X_i and X_j $(i \neq j)$ joined by an edge whenever some vertex in X_i is adjacent (in G) to some vertex in X_j.

Pseudo-code: InvariantTriangleByCell

```
Data: A graph G, a cell C = {v_1, ..., v_k}
      N(u) denotes the ordered neighbor list of u in G
Result: The number of triangles contained in C
count ← 0
for each u ∈ C do
    for each v ∈ N(u) ∩ C such that v > u do
        for each w ∈ N(u) ∩ N(v) ∩ C such that w > v do
            count ← count + 1
        end for
    end for
end for
return count
```

For counting the number of triangles in a cell, the most classical approach enumerates all triplets of vertices (u, v, w) from the cell and checks for each one whether all three edges exist, running in $O(n^3)$ where n is the number of vertices in the cell. A faster approach exploits the sorted neighbor lists $N(v)$: for each vertex $u \in C$, one iterates over its neighbors v in the cell with $v > u$, then looks for common neighbors w of both u and v still in the cell with $w > v$. Since the neighbor lists are sorted, each such lookup runs in $O(d)$ where d is the largest number of neighbors that any vertex in the cell has, yielding an overall complexity of $O(n \cdot d^2)$.

2. *two paths by cell*: for each cell of the partition, we examine every path of length two starting from a vertex of that cell, and weight each such walk by the cell it reaches (which may be the starting vertex itself). Here, each cell of the partition is represented by an integer, namely its index within the partition, denoted cellIndex(v) in the code. The invariant of a cell C is then obtained by summing, over every vertex $v \in C$ and every vertex w reachable from v by a path of length two, the index of the cell containing w. This yields a single integer representing the invariant of the cell. This method is adapted directly from the *Nauty* user's guide [47]; since only its high-level description was available to us, rather than the corresponding source code, it had to be reconstructed and generalised to fit our cell-based framework.

pseudo-code of InvariantTwoPathsByCell

```
Data: A graph G, a cell C = {v_1, ..., v_k}
Result: the invariant value of the cell C
count ← 0
for all vertices v ∈ C do
    for all neighbors u ∈ N(v) do
        count ← count + Σ_{w∈N(u)} cellIndex(w)
    end for
end for
return count
```

The computation of this invariant is divided into two steps. First, for every vertex $u \in V$, we precompute the value $S(u) = \sum_{w \in N(u)} \text{cellIndex}(w)$ by scanning the adjacency list of each vertex once. Since every edge is traversed at most twice, this preprocessing step requires $O(n+m)$ time, where n is the number of vertices and m

is the number of edges. Once these values have been computed, the invariant of a cell C is obtained by iterating over every vertex of the cell and summing the precomputed values of its neighbors. The running time for one cell is therefore $O\big(\sum_{v\in C}\deg(v)\big)$. Since the cells form a partition of the vertex set, each vertex is processed exactly once when computing the invariant for all cells, yielding an overall running time of $O(n+m)$.

3. *triple by cell*: It is also an invariant method adapted directly from the *Nauty* user's guide [47]. However, since no source code was available, we had to reconstruct the method from its description. The idea is to consider, for each cell, every triple ofvertices $\{u,v,w\}$ it contains. For each triple, we count the number of vertices x of the graph that are adjacent to an odd number of the vertices of $\{u,v,w\}$ (that is, adjacent to exactly one or exactly three of them); this count is the weight of the triple, and it is added to the counter of each of the three vertices u, v and w. The final invariant of the cell is the sum of these counters over every vertex it contains. The main drawback of this method is that it is computationally expensive, as it requires enumerating every triple of vertices in each cell, which can be prohibitive for large cells.

pseudo-code of InvariantCellTriplesByCell

```
1: Data: A graph G, a partition π = (C_1, ..., C_k)
2: Result: the invariant value of each cell of π
3: I ← (0, ..., 0) of size k
4: for all cells C_i ∈ π do
5:     for all triples of vertices {u, v, w} ⊆ C_i do
6:         weight ← 0
7:         for all vertices x ∈ V(G) do
8:             if |N(x) ∩ {u, v, w}| is odd then
9:                 weight ← weight + 1
10:            end if
11:        end for
12:        I[i] ← I[i] + weight
13:    end for
14: end for
15: return I
```

For a cell C_i of size $|C_i|$, the number of triples $\{u,v,w\}\subseteq C_i$ is $\binom{|C_i|}{3} = O(|C_i|^3)$, and computing the weight of a single triple requires scanning all n vertices of G. Evaluating the invariant on C_i therefore costs $O(|C_i|^3\cdot n)$. Since this is computed for every cell of π, and $\sum_i |C_i| = n$, the worst case gives an overall cost of $O(n^4)$. In practice, cells are usually much smaller once the standard refinement has run, keeping the actual cost far below this bound.

4.2.3 Pruning strategies

To avoid generating unnecessary information in our search tree,which would result in an overly large tree, we aim to construct the most optimised tree possible. To achieve this, we can apply *pruning* techniques to the tree: eliminating redundant branches so that only a fraction of the original tree needs to be generated. We can define two types of pruning strategies to optimize our construction:

$\mathcal{P}_A(\sigma, \sigma')$: We remove $T(G, \pi_0, \sigma')$ whenever σ, σ' are distinct nodes of $T(G, \pi_0)$ satisfying $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) > \phi(G, \pi_0, \sigma')$.

$\mathcal{P}_B(\sigma, \sigma')$: We remove $T(G, \pi_0, \sigma')$ whenever σ, σ' are distinct nodes of $T(G, \pi_0)$ satisfying $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) \neq \phi(G, \pi_0, \sigma')$.

$\mathcal{P}_C(\sigma, g)$: We remove $T(G, \pi_0, \sigma')$ whenever $g \in \mathrm{Aut}(G, \pi_0)$ and $\sigma < \sigma'$ are nodes of $T(G, \pi_0)$ such that $\sigma^g = \sigma'$.

Informally, each pruning method serves a distinct purpose. The method $\mathcal{P}_A$ discards branches whose invariant cannot reach the maximal value: at each level of the tree, we keep track of the best invariant found so far, and any other node at the same level with a strictly worse invariant has its branch pruned. The method $\mathcal{P}_B$ is somewhat more subtle. Rather than comparing the current node to the best node found so far, it is compared to a reference node, taken as the node at the same level lying on the first path leading to the first leaf. If the two invariants differ, the current subtree is pruned. Here, the goal is not really to find the best invariant, but rather to uncover new automorphisms. Combining the two methods, a branch is pruned only when both $\mathcal{P}_A$'s and $\mathcal{P}_B$'s conditions are satisfied. In other words, the current node is pruned if its invariant is strictly worse than the best invariant found so far and if it differs from the invariant of the reference node. Finally, the method $\mathcal{P}_C$ exploits automorphisms already discovered during the search. Suppose an automorphism $g \in \mathrm{Aut}(G, \pi_0)$ maps the current node σ to another node σ^g that has already been explored. Since the corresponding subtrees are isomorphic, exploring both would be redundant; the subtree rooted at σ^g can therefore be discarded.

Despite these pruning strategies, we must ensure that our search tree, along with the node invariant function, remains functionally correct. To this end, we require a theorem that guarantees the validity of the overall construction.

Theorem 5. *Consider any $G \in \mathcal{G}$ and $\pi_0 \in \Pi$*

- *Suppose any sequence of operations of the form $\mathcal{P}_A(\sigma, \sigma')$ or $\mathcal{P}_C(\sigma, g)$ are performed. Then there remains at least one leaf σ_1 with $\phi(G, \pi_0, \sigma_1) = \phi^*(G, \pi_0)$.*
- *Let σ_0 be some fixed leaf of $T(G, \pi_0)$. Suppose any sequence of operations of the form $\mathcal{P}_B(\sigma, \sigma')$ $\mathcal{P}_C(\sigma'', g)$ are performed, where $\phi(G, \pi_0, \sigma'') \neq \phi(G, \pi_0, [\sigma_0]_{|\sigma''|})$. Let $g_1, \ldots, g_k$ be the automorphisms used in the operations $\mathcal{P}_C$ that were performed, and let $A = \{g \in \mathrm{Aut}(G, \pi_0)$: σ_0^g is a remaining leaf$\}$. Then $\mathrm{Aut}(G, \pi_0)$ is generated by $\{g_1, \ldots, g_k\} \cup A$.*

Proof. To prove the first point, let σ_1 denote the lexicographically smallest leaf satisfying $\phi(G, \pi_0, v_1) = \phi^*(G, \pi_0)$. Operation $\mathcal{P}_A$ only removes branches whose invariant cannot reach the maximal value, while operation $\mathcal{P}_C$ always preserves the lexicographically smallest representative of an automorphism class. Consequently, σ_1 cannot be removed. Therefore, at least one leaf with maximal invariant always remains after any sequence of pruning operations.

For the second point, let $g \in \mathrm{Aut}(G, \pi_0)$ and consider the corresponding leaf σ_0^g.

If σ_0^g is still present after pruning, then by definition $g \in A$.

Otherwise, σ_0^g must have been removed by an operation $\mathcal{P}_C(\sigma'', g_i)$. It cannot be removed by $\mathcal{P}_B(\sigma', \sigma)$ because it only removes leaves inequivalent to σ_0. This implies that there

exists a lexicographically smaller equivalent leaf, namely

$$\sigma_0^{gg_i^{-1}} < \sigma_0^g.$$

If this new leaf has also been removed, the same reasoning applies. Repeating this process produces a strictly decreasing sequence of equivalent leaves

$$\sigma_0^g > \sigma_0^{gg_i^{-1}} > \sigma_0^{gg_i^{-1}g_j^{-1}} > \cdots$$

Since the lexicographic order on the finite set of leaves is well-founded, this sequence must eventually terminate at a remaining leaf. Hence, there exists $h \in \langle g_1, \ldots, g_k \rangle$ such that σ_0^{gh} is a remaining leaf. By definition of A, we have $gh \in A$, and therefore $g = (gh)h^{-1}$ where $gh \in A$ and $h^{-1} \in \langle g_1, \ldots, g_k \rangle$.

Since g was arbitrary, every automorphism belongs to the group generated by

$$\{g_1, \ldots, g_k\} \cup A,$$

which proves the result. □

Informally, This theorem and its proof establish that pruning can never eliminate every leaf realizing the maximal invariant: at least one such leaf always survives. The second part of the theorem further shows that the full automorphism group $\mathrm{Aut}(G, \pi_0)$ remains recoverable despite pruning: it is generated by the witnesses $g_1, \ldots, g_k$ collected along the $\mathcal{P}_C$ operations, together with the automorphisms in A corresponding to the surviving leaves in the orbit of a fixed reference leaf σ_0. In other words, even though the vast majority of leaves equivalent to σ_0 are discarded during the search, the automorphisms they correspond to are never lost: each one can be recovered as a product of an element of A and an element of the subgroup generated by $g_1, \ldots, g_k$.

The $\mathcal{P}_A$ pruning strategy is relatively straightforward to implement. At each level of the search tree, we simply compare the node invariant of the node currently being explored with the best node invariant encountered so far at the same level. If the invariant of the current node is smaller than the best one found previously, then the corresponding subtree can safely be discarded.

The $\mathcal{P}_B$ strategy is slightly more involved. Instead of comparing the current node with the best node encountered so far, we compare its node invariant with that of a reference node. This reference node is chosen as the node at the same level lying on the first path leading to the first leaf. If the two node invariants differ, the current subtree is pruned. Conversely, branches whose node invariants coincide with those of the reference path remain candidates for discovering new automorphisms.

Finally, the $\mathcal{P}_C$ strategy exploits the automorphisms that have already been discovered during the search. Suppose that an automorphism $g \in \mathrm{Aut}(G, \pi_0)$ maps the current node σ to another node σ^g that has already been explored. Since the corresponding subtrees are isomorphic, exploring both of them would be redundant. Consequently, the subtree rooted at σ^g can be discarded.

Implementing this strategy is considerably more challenging than implementing $\mathcal{P}_A$ or $\mathcal{P}_B$, and several approaches have been proposed over the years. In the earliest versions of *Nauty*, pruning relied solely on the automorphisms discovered during the search. More recent

versions additionally provide an optional *Random Schreier* method, which *Nauty* adopted in place of a deterministic alternative for its greater efficiency, and which constructs a randomised approximation of the automorphism group from the automorphisms already found.This often enables more aggressive pruning, although its effectiveness depends on the input graph; for some graph families it provides little or no improvement and may even introduce additional overhead.

In this work, we implement the original approach, which relies only on the automorphisms discovered during the search. To represent and manipulate the automorphism group efficiently, we use the *Schreier-Sims algorithm*, which constructs a base and strong generating set (BSGS) together with the associated stabiliser chain, allowing efficient orbit computations and pruning decisions.

Schreier-Sims Algorithm

A permutation group $G \leq S_n$ given by a set of generators[3] may contain an enormous number of elements, making it impractical to work with by listing them explicitly. The Schreier-Sims algorithm avoids this by building a compact representation of G, from which the order $|G|$, membership testing, and orbit computations can all be obtained efficiently.

The idea is to shrink the group step by step. Starting from G, we choose a point b_1 and restrict our attention to the permutations of G that happen to fix it: this smaller group is the *stabiliser* of b_1. We then choose a second point b_2 and repeat the same reduction inside this smaller group, and so on, until only the identity permutation is left. The sequence of chosen points $b_1, \ldots, b_k$ is called a *base*. Writing $G^{(0)} = G$ and, for $i \geq 1$, $G^{(i)} = \mathrm{Stabiliser}_{G^{(i-1)}}(b_i)$, this produces a descending chain of subgroups

$$G = G^{(0)} \geq G^{(1)} \geq \cdots \geq G^{(k)} = \{\mathrm{id}\}.$$

Informally, each group in the chain is smaller than the one before it, but its generators let us climb back up and reconstruct the group above it.

This reduction is made computationally cheap by two related notions. The *orbit* of the current base point is the set of points it can be mapped to by the current group; it can be computed by a breadth-first search using only the group's generators. This same search yields a *transversal*, that is, for every point in the orbit, a permutation of the current group mapping the base point to it. Combining the transversal with the current generators, through the so-called *Schreier generators*, produces a generating set for the next, smaller stabiliser, without ever enumerating the elements of the group itself. These Schreier generators are constructed from the generators of the current group and the transversal, and they need not be among the original generators. Repeating this process at each level produces generators for the successive stabilisers.

This process gives a base together with a generating set for each successive stabiliser. The generators obtained at the different levels form a *strong generating set*, since they provide generators for the successive stabilisers in the chain. The base together with this strong generating set is called a *Base and Strong Generating Set* (BSGS). It also gives an immediate way to compute the order $|G|$: by the orbit-stabiliser theorem [64], $|G|$ is simply the product of the orbit sizes obtained at each step,

[3] A *generating set* of G is a subset $S \subseteq G$ such that every element of G can be written as a product of elements of S; we then write $G = \langle S \rangle$.

$$|G| = \prod_{i=1}^{k} o_i,$$

where o_i is the size of the orbit of b_i computed at step i.

The polynomial-time nature of this construction was not established immediately: Sims's original description came with no formal running-time analysis. Furst, Hopcroft and Luks were the first to show that a version of the algorithm runs in polynomial time, namely $O(n^6)$, where n is the number of elements [24]. This bound was then improved by Knuth to $O(n^5)$ [38], and later implementations relying on Schreier vectors, as described by Seress, bring the running time down to $O(n^2 \log^3 |G| + tn \log |G|)$ [58] where t is the number of generators.

To better understand the construction, we consider the graph $K_{1,3}$. We build the BSGS associated with $\mathrm{Aut}(K_{1,3})$ using the base $(b_1 = 1, b_2 = 2)$.

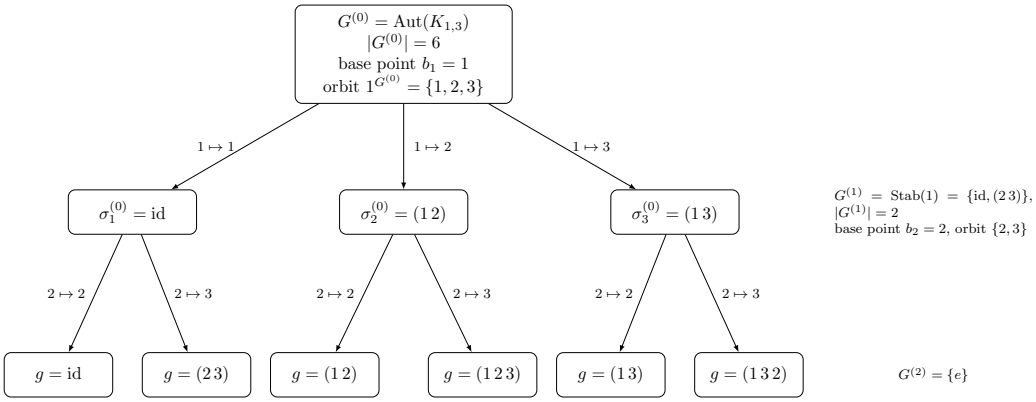


Figure 4.4: Tree associated with the BSGS of $\mathrm{Aut}(K_{1,3})$ with base $(b_1 = 1, b_2 = 2)$.

We observe that each edge at level i carries a transversal element $\sigma_p^{(i)}$ for the orbit of b_{i+1} under $G^{(i)}$; a leaf is the element of G obtained by composing these labels along its root-to-leaf path. Since the successive orbits have sizes 3, 2, 1, the tree has $3 \times 2 \times 1 = 6$ leaves. This is exactly the elements of $\mathrm{Aut}(K_{1,3})$.

Schreier-Sims for *Nauty*

Let G be the input graph and π_0 its initial partition. Unlike the setting described above, where Schreier-Sims is applied to a group given by a known generating set, the automorphism group $\mathrm{Aut}(G, \pi_0)$ is not known in advance here: its generators are discovered incrementally, one by one, over the course of the search.

Nauty performs a depth-first traversal of the search tree. Each leaf σ corresponds to a permutation $R(G, \pi_0, \sigma)$, obtained by applying the refinement function along the individualisation sequence σ. Whenever a newly reached leaf σ' produces the same invariant as the first leaf σ_0 reached by the search, the permutation

$$g = R(G, \pi_0, \sigma')\, R(G, \pi_0, \sigma_0)^{-}1$$

is an automorphism of G, and is added to the currently known automorphism group.

These discovered automorphisms are the generators from which the group structure is built. The automorphism g is then sifted through the existing stabiliser chain, correcting its action on each base point successively. It is either absorbed by an existing level, in which case it provides no new information, or it is identified as a new strong generator and enriches the chain.

Each automorphism discovered at a leaf is propagated to the ancestor nodes for which it remains relevant. The accumulated group information is then used when selecting the next vertex to individualise. Once a target cell has been chosen, its vertices are partitioned into orbits under the currently known automorphism group. Two vertices belonging to the same orbit necessarily lead to isomorphic subtrees, since they are related by an automorphism already known to the algorithm. Consequently, it is sufficient to explore a single representative of each orbit, while all other vertices can be pruned without exploration.

This is a direct application of the pruning operation $\mathcal{P}_C$.

To get a concrete idea of what this looks like in code, here is a simplified pseudo-code of the Schreier-Sims method for *Nauty*.

Pseudo-code of Schreier-Sims method for Nauty

```
1: Data: graph G, initial partition π_0, reference leaf σ_0, current base B
2: Result: BSGS
3: for each leaf σ' reached by the depth-first search do
4:     if φ(G, π_0, σ') = φ(G, π_0, σ_0) then
5:         g ← R(G, π_0, σ') R(G, π_0, σ_0)^{-1}          ▷ candidate automorphism
6:         if g ≠ id then
7:             InsertGenerator(g, level 1)
8:         end if
9:     end if
10: end for

11: procedure InsertGenerator(g, level i)
12:     if B has no point at level i then
13:         choose a new base point b_i moved by g
14:     end if
15:     add g to the strong generators at level i
16:     GrowOrbit(i)                  ▷ updates the orbit and transversal at level i
17:     Closestabiliser(i)            ▷ Schreier generators, recurses if needed
18: end procedure
```

Random Schreier method

The main difference between the classical Schreier-Sims algorithm seen previously and the Random Schreier method lies in the way the BSGS is constructed. Instead of deterministically maintaining an exact base and strong generating set from every newly discovered automorphism, the Random Schreier method adopts a probabilistic approach. It repeatedly generates random products of the automorphisms already known and inserts these permutations into the Schreier-Sims structure.

This randomization trades a deterministic guarantee for speed: the resulting approximation of the BSGS is correct with high probability rather than with certainty, in the Monte Carlo sense. Babai, Cooperman, Finkelstein, Luks and Seress showed that random products of the known generators emulate genuinely random group elements closely enough to yield a nearly optimal $O(n^3 \log^c n)$ membership test [5].

The objective is not to compute the complete BSGS as early as possible, but rather to obtain a sufficiently rich approximation of the subgroup generated by the discovered automorphisms. This approximation often provides accurate orbit information much earlier during the search, allowing the $\mathcal{P}_C$ pruning strategy to eliminate redundant branches sooner. Consequently, the search tree may become significantly smaller.

Since the method relies on randomization, its effectiveness depends on the input graph. For many highly symmetric graphs, it substantially improves the pruning process. However, for graphs with few automorphisms, the additional computations required to generate and process random group elements may outweigh the benefits. For this reason, the Random Schreier method is implemented as an optional feature in *Nauty*.

Pruned Search Tree Construction

To illustrate how the $\mathcal{P}_C$ pruning strategy operates in practice, we return to our search tree for $K_{1,3}$ (c.f. Fig. 4.3) and apply the $\mathcal{P}_C$ pruning strategy.

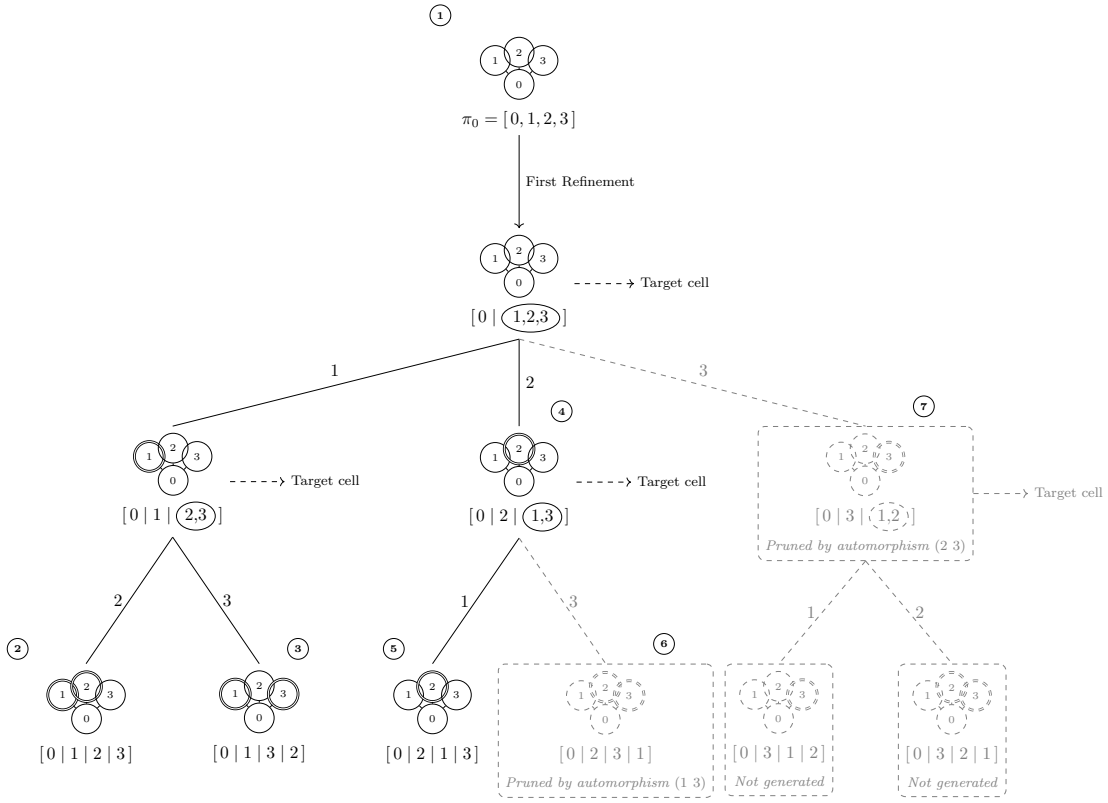


Figure 4.5: Search tree for $K_{1,3}$ with the $\mathcal{P}_C$ pruning strategy applied.

To better understand how this strategy works, we will walk through the construction of the pruned search tree step by step, highlighting the discovered automorphisms and the branches that are pruned.

① We start from the original graph.

② The first leaf reached, obtained by individualizing 1, then 2, then 3, is taken as the reference leaf against which every subsequent leaf is compared.

③ Comparing the second leaf to the reference leaf yields the automorphism (2 3).

④ When vertex 2 is individualised, none of the automorphisms found so far allow pruning any of its children. The target cell $\{1, 3\}$ is therefore kept in full.

⑤ Individualizing 1 next yields a leaf whose comparison with the reference leaf gives the automorphism (1 2). Combined with (2 3) found earlier, this closes the automorphism group to S_3, which in particular contains (1 3).

⑥ This new automorphism is then used to prune the remaining leaf of that branch.

⑦ Likewise, (2 3) shows that individualizing vertex 3 first is equivalent to individualizing vertex 2 first. The corresponding branch is therefore pruned entirely, without ever being explored.

4.2.4 Final Search Tree Construction

We can put in practice our different pruning strategies on our search tree. *Nauty* will generate its tree in depth-first order. The lexicographically least leaf σ_1 is kept and if the canonical labelling is sought (rather than just the automorphism group), the leaf σ^* with the greatest invariant discovered so far is also kept. A non-discrete node σ is pruned if neither $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, [\sigma_1]_{|\sigma|})$ or $\phi(G, \pi_0, \sigma) \geq \phi(G, \pi_0, [\sigma^*]_{|\sigma|})$. Such operations correspond to $\mathcal{P}_A(\sigma^*, \sigma)$ and $\mathcal{P}_B(\sigma_1, \sigma)$. We can therefore conclude that only a subset of the tree will be generated, and that the generated leaves are those that may be "better" than the leaves already discovered.

Pseudo-code of canonical labelling

```
1:  Data: A graph G = (V, E)
2:  Result: The canonical form of G
3:  π ← ({0, 1, ..., |V| − 1})
4:  π ← F(G, π, a list of all the cell of π)
5:  π* ← π                                          ▷ π* the best partition
6:  π_1 ← ()                                        ▷ π_1 the partition of first leaf
7:  σ ← (),    σ* ← ()                              ▷ current and best vertex sequences
8:  procedure NAUTY(G, π)
9:      if π is discrete then
10:         if π* is not discrete then              ▷ no leaf found yet
11:             π* ← π, σ* ← σ, π_1 ← π
12:             BSGS ← Base (σ)                     ▷ initialize Base
13:         else if φ(G, π, σ) > φ(G, π*, σ*) then
14:             π* ← π, σ* ← σ                      ▷ better leaf found
15:         end if
16:         return
17:     end if
18:     if π* is discrete and φ(G, π, σ) < φ(G, π*, σ*) and φ(G, π, σ) ≠ φ(G, π_1, σ_1) then
19:         return          ▷ Pruning P_A + P_B: cannot beat π*, nor tie π_1
20:     end if
21:     Cell ← T(G, π, σ)
22:     R ← orbit representatives of Cell           ▷ Pruning P_C
23:     for each ν ∈ R do
24:         π_ν ← I(π, ν)
25:         π_ν ← F(G, π_ν, {ν})
26:         Append ν to σ
27:         NAUTY(G, π_ν)
28:         Remove last entry from σ
29:     end for
30: end procedure
31: g ← canonical vertex ordering induced by π*
32: return G^g                  ▷ Apply g to G to get the canonical form
```

Chapter 5

Testing and Results

This chapter presents the experiments conducted to validate the correctness of our implementation, evaluate its performance on different classes of input graphs, and compare its behaviour with the original *Nauty* implementation.

The source code of our implementation is available on GitHub.

5.1 Input and output formats

Before presenting the experimental results, we briefly describe the input and output formats supported by our implementation.

Although there is no universally accepted graph encoding for graph isomorphism algorithms, our implementation follows the convention established by *Nauty* and uses the *graph6* format [46]. This compact representation is widely used in the graph isomorphism community and provides an efficient way to store, exchange, and generate large collections of graphs while remaining fully compatible with existing tools.

The program supports five execution modes:

- *iso:* compares two graphs and determines whether they are isomorphic.
- *canonical:* computes and outputs the canonical form of the input graph in graph6 format.
- *all:* computes the canonical form of the input graph and, for every leaf explored during the search, recomputes its canonical form independently to verify that all search paths produce the same canonical labelling. This mode is intended primarily for debugging and validation.
- *aut:* computes the canonical form of the input graph together with its automorphism group.
- *file:* reads a collection of graphs encoded in graph6 format from a file and computes the canonical form of each graph. This mode is particularly useful for large-scale experiments and performance evaluation.

The user can also choose the node invariant used during canonical labelling. The currently available invariants are *triangle*, corresponding to InvariantTriangleByCell, *paths*,

corresponding to InvariantTwoPathsByCell, and *triple*, corresponding to InvariantCellTriplesByCell.

5.2 Correctness testing

Before evaluating the performance of our implementation, we first verified its correctness through a collection of unit and integration tests covering all major components of the algorithm.

Unit tests. Each core component of the implementation was tested independently to ensure that it produced the expected output. The refinement procedure was validated on manually constructed partitions for which the refined partitions were known in advance. Likewise, the target cell selection procedure was tested on several representative partitions to verify that it consistently selected the expected target cell. Finally, each node invariant was evaluated on a variety of partitions to ensure that the computed invariant matched the expected value.

Canonical labelling. The correctness of the complete canonical labelling algorithm was verified using the *all* execution mode. In addition, the *file* mode was used on collections of graphs to verify that every input graph was assigned the expected canonical form. As a representative example, the graph $K_{1,3}$ was used throughout the development to validate the behaviour of the search tree and to ensure that the computed canonical labelling agreed with the expected result.

Pruning procedures. Each pruning strategy was validated independently. For pruning rule $\mathcal{P}_A$, we verified that every node whose invariant was strictly worse than the best invariant found so far was correctly discarded. For pruning rule $\mathcal{P}_B$, we verified that every node whose invariant differed from that of the current reference node was correctly pruned. Finally, pruning rule $\mathcal{P}_C$ was validated using the search tree shown in Figure 4.5, where every branch predicted by the theory to be removable was successfully eliminated.

Automorphism groups. The correctness of the computed automorphism groups was verified by comparing our implementation against *Nauty*. Using the *file* execution mode, we computed the automorphism group of each graph in a collection of test instances and compared the results with those produced by *Nauty*. For every graph, both algorithms produced automorphism groups of identical order, providing strong evidence for the correctness of our implementation.

5.3 Performance testing

All the tests presented in the remainder of this chapter are carried out on several families of graphs: trees, sparse graphs[1], random graphs $G(n, 1/2)$, bipartite graphs, cycles[2], regular graphs[3], dense graphs[4], and hypercube graphs[5]. It will allowing us to observe the behavior of the invariants across a wide range of structures. For each graph family, several datasets are used, with all graphs within a given dataset sharing the same number of vertices.

The datasets were generated using `genrang` [47], the random graph generation utility included with *Nauty* (version 2.9.3), using family-specific options (e.g.`-P` for edge probability, `-t` for random trees, `-r` for regular graphs (a fixed degree of 3 was used across all sizes), and `-e` for a fixed edge count). Each dataset contains 100 randomly generated graphs from the same graph class, with graph sizes ranging from 10 to 1,000 vertices. Hypercube graphs are an exception to this range: since Q_d has exactly 2^d vertices, this family is instead tested for dimensions $d = 1$ to 10, corresponding to graph sizes of $2, 4, 8, \ldots, 1024$ vertices. As Q_d is unique up to isomorphism for a given dimension, it was instead generated once using `genspecialg` (option `-Q`) [47] and then relabeled 1,000 times using `ranlabg` [47] to obtain 1,000 distinct labelled instances per dimension. To ensure that all experiments are reproducible and performed on exactly the same inputs, the datasets were generated only once prior to the experiments and stored as graph6 files. Consequently, every benchmark reported in this chapter is performed on the same collection of graphs.

Several experiments are then performed on these datasets. First, we measure and compare the running time of all our invariants across the different datasets, and against the default version of *Nauty*. A Wilcoxon test is used to statistically compare the invariants against one another. We also measure the pruning ratio achieved by each invariant on each graph family. Finally, we evaluate our version of *Nauty* on the same graphs with pruning disabled, in order to isolate the effect of the pruning rules themselves.

All running times are measured in wall-clock (real) time. Experiments were conducted on a Lenovo IdeaPad 3 15ITL6 laptop equipped with an 11th generation Intel Core i3–1115G4 processor (x86_64 architecture) running at 3.00 GHz, 7.6 GiB of RAM, and Ubuntu 24.04.1 LTS (kernel version 6.8.0–51-generic). This machine was dedicated exclusively to the experiments, with no other process running concurrently, and no time limit was imposed on the executions.

Legend. Throughout the following experiments, the horizontal axis always denotes the number of vertices and the vertical axis the average execution time (in ms); each invariant is represented by a distinct symbol and colour. Since these correspondences are the same across all figures, we introduce them once below rather than repeating them under every graph.

▲ triangle ♦ paths ● triple × *Nauty*

[1] A graph where the number of edges is on the order of the number of vertices, placing it near the threshold for the emergence of a giant connected component.

[2] A graph consisting of a single cycle with at least three vertices, where each vertex is connected to exactly two other vertices.

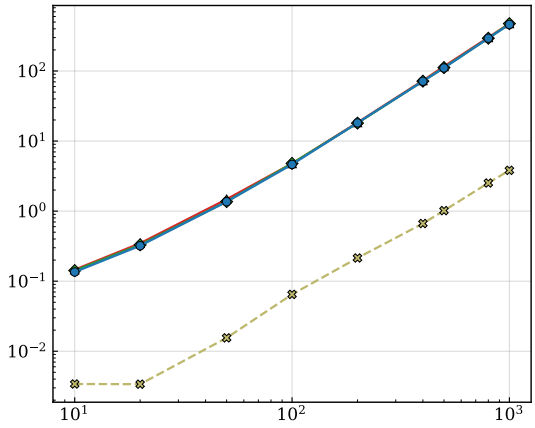
[3] A graph where every vertex has the exact same number of neighbors.

[4] A graph where the number of edges is close to the maximum possible number of edges.

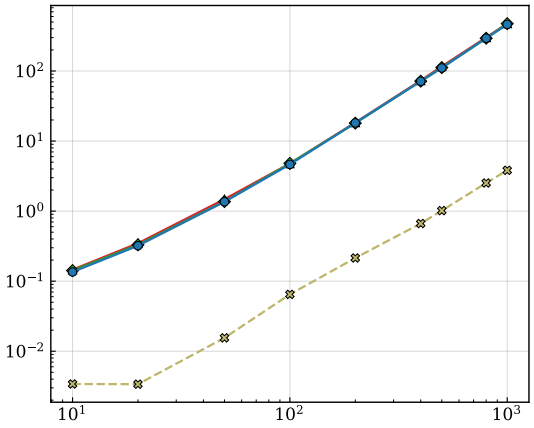
[5] A graph denoted Q_d whose vertices are represented by a binary strings of length d, with an edge between two vertices whenever they differ in exactly one bit.

5.3.1 Time performance

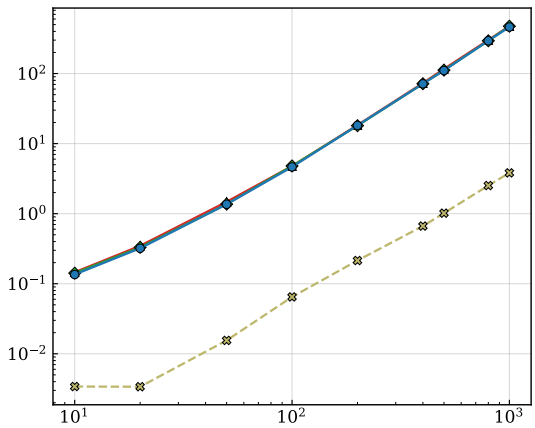
Without prunning



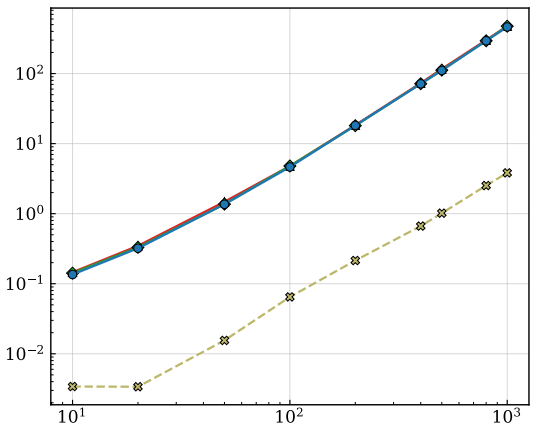
Random graphs $G(n, 1/2)$



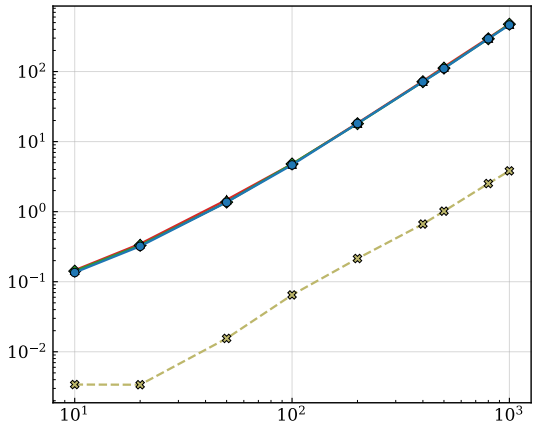
Trees



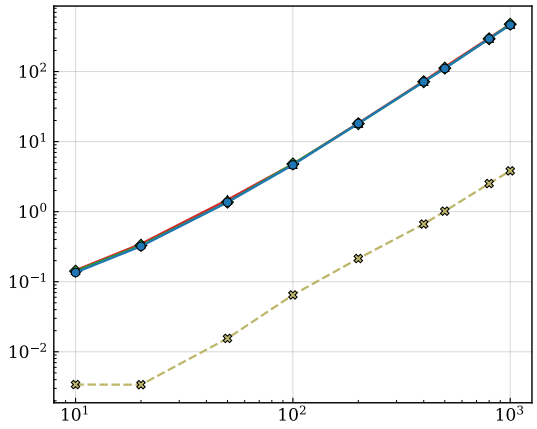
Regular graphs



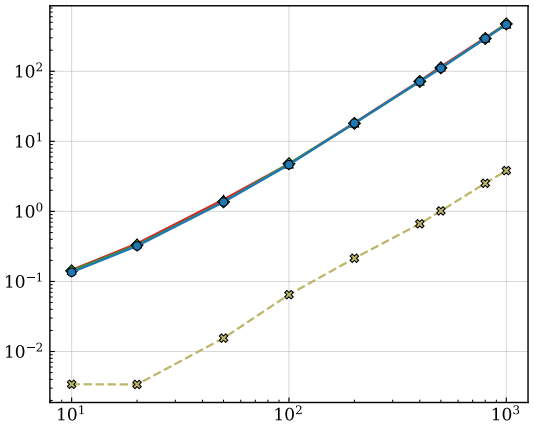
Bipartite graphs



Dense Graphs



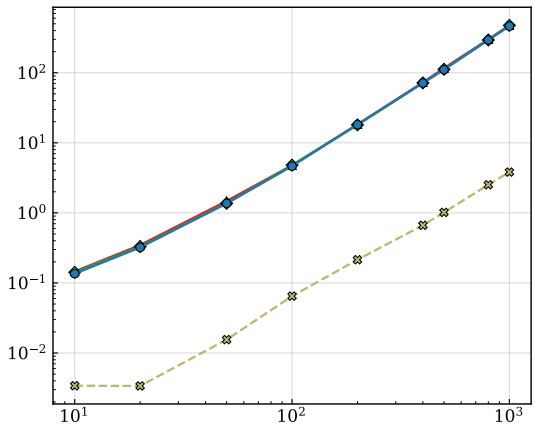
Sparse Graphs



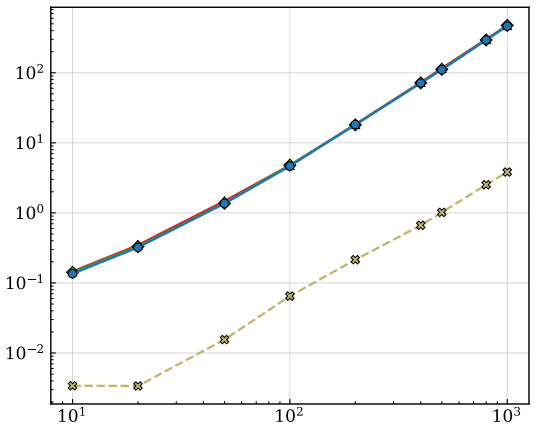
Cycles

Discussion of the results

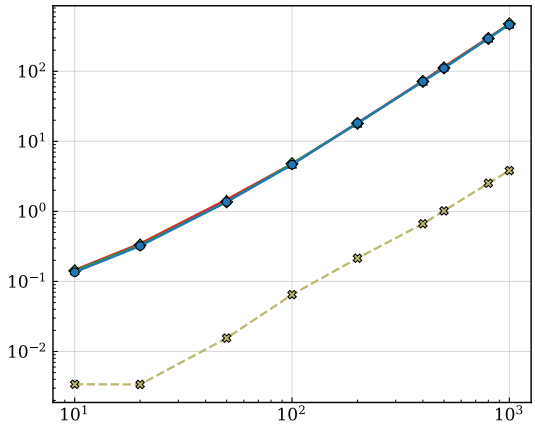
With prunning



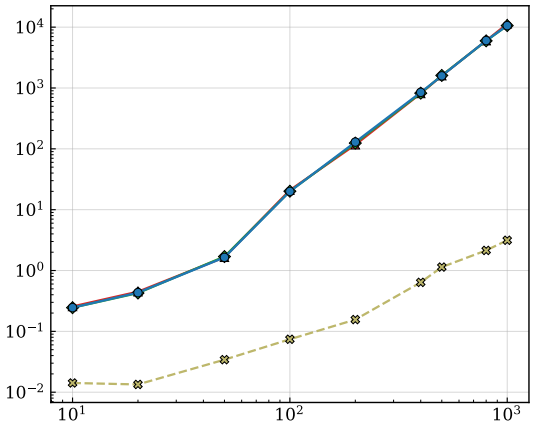
Random graphs $G(n, 1/2)$



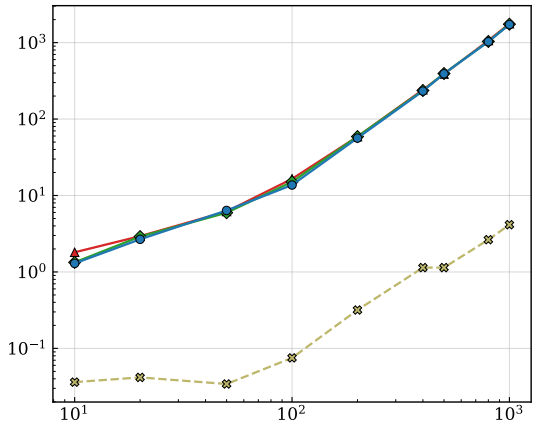
Trees



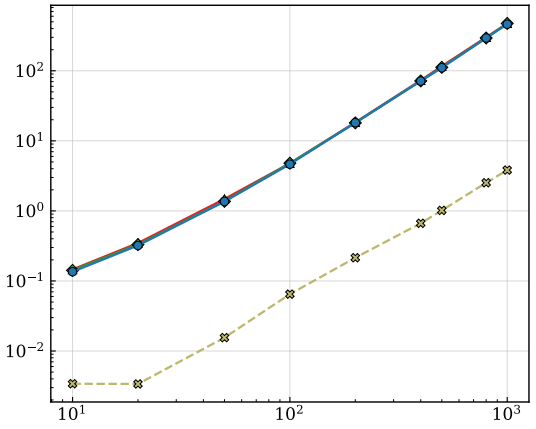
Regular graphs



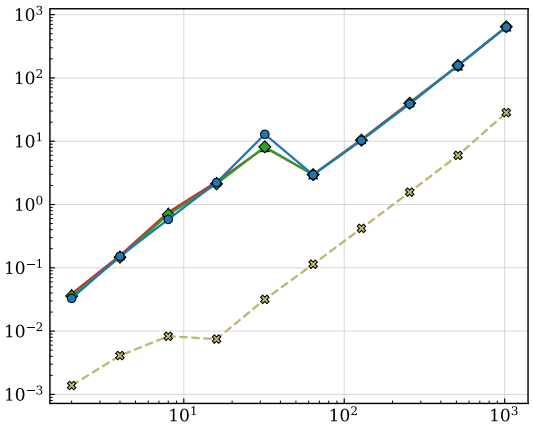
Bipartite graphs



Dense Graphs



Sparse Graphs



Hypercube graphs

Wilcoxon signed-rank test

Table 5.1: Average pruning statistics.

Vertices	Nodes	Leaves	Pruned	P_A	P_B	P_C
	1.4	1.2	0.0	0.00%	0.00%	100.00%
10	1.4	1.2	0.0	0.00%	0.00%	100.00%
	1.4	1.2	0.0	0.00%	0.00%	100.00%
	1.0	1.0	0.0	0.00%	0.00%	0.00%
50	1.0	1.0	0.0	0.00%	0.00%	0.00%
	1.0	1.0	0.0	0.00%	0.00%	0.00%
	1.0	1.0	0.0	0.00%	0.00%	0.00%
	1.0	1.0	0.0	0.00%	0.00%	0.00%
200	1.0	1.0	0.0	0.00%	0.00%	0.00%
	1.0	1.0	0.0	0.00%	0.00%	0.00%

Within each block of three rows, results are given in order for the *triangle*, *paths*, and *triple* invariants.

Discussion of the results

La première chose ici que nous pouvons observer c'est qu'il est très très compliqué de battre l'implémentation originale de *Nauty* sur tous les classes de graphes. Comme énoncé, l'implémentation originale de *Nauty* est extrêmement optimisée et a été développée et testée pendant de nombreuses années. Il est donc naturel que notre implémentation, bien que correcte et fonctionnelle, ne parvienne pas à surpasser *Nauty* en termes de performance sur toutes les classes de graphes. Ça reste tout de même intéressant de voir la différence entre nos 2 implémentations. On remarque surtout que l'implémentation originale de *Nauty* reste beaucoup plus performante sur les graphes sparse, trees et les graphes réguliers car on remarque vraiment une énorme différence dans les temps moyens d'éxecutions.

Ensuite, si on se concentre sur notre implémentation et qu'on compare les 3 invariants, nous pouvons voir que pour une première partie de classe de graphes, les invariants reste assez proche les uns des autres, mais pour les trees, les graphes sparse et les graphes réguliers, on remarque que l'invariant *triple* est beaucoup plus coutant que les 2 autres. Il y a une vraie différence dans leur temps d'execution. Ce qui montre que les 2 autres invariants restent mieux "par défaut" pour ces classes de graphes.

Le Wilcoxon test ne fais que confirmer ces propos.

5.3.2 Pruning performance

Vertices	Nodes	Leaves	P_A	P_B	P_C
10	1.5	1.2	0%	0%	100%
	1.5	1.2	0%	0%	100%
	1.5	1.2	0%	0%	100%
20	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
50	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
100	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
200	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
400	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
500	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
800	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
1000	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%

Random graphs $G(n, 1/2)$

Vertices	Nodes	Leaves	P_A	P_B	P_C
10	1.5	1.2	0%	0%	100%
	1.5	1.2	0%	0%	100%
	1.5	1.2	0%	0%	100%
20	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
50	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
100	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
200	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
400	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
500	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
800	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
1000	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%
	1.0	1.0	0%	0%	0%

Random graphs $G(n, 1/2)$

Vertices	Invariant	Nodes	Leaves	Pruned	P_A	P_B	P_C
10	triangle	1.4	1.2	0.0	0.00%	0.00%	100.00%
10	paths	1.4	1.2	0.0	0.00%	0.00%	100.00%
10	triple	1.4	1.2	0.0	0.00%	0.00%	100.00%
50	triangle	1.0	1.0	0.0	0.00%	0.00%	0.00%
50	paths	1.0	1.0	0.0	0.00%	0.00%	0.00%
50	triple	1.0	1.0	0.0	0.00%	0.00%	0.00%
200	triangle	1.0	1.0	0.0	0.00%	0.00%	0.00%
200	paths	1.0	1.0	0.0	0.00%	0.00%	0.00%
200	triple	1.0	1.0	0.0	0.00%	0.00%	0.00%

Table 5.3: Average pruning statistics.

Vertices	Nodes	Leaves	P_A	P_B	P_C
	3.9	2.1	0.00%	0.00%	100.00%
10	3.9	2.1	0.00%	0.00%	100.00%
	3.9	2.1	0.00%	0.00%	100.00%
	1.1	1.1	0.00%	0.00%	100.00%
20	1.1	1.1	0.00%	0.00%	100.00%
	1.1	1.1	0.00%	0.00%	100.00%
	1.0	1.0	0.00%	0.00%	0.00%
50	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	3.4	2.0	0.00%	0.00%	100.00%
100	3.4	2.0	0.00%	0.00%	100.00%
	3.4	2.0	0.00%	0.00%	100.00%
	3.4	2.0	0.00%	0.00%	100.00%
200	3.4	2.0	0.00%	0.00%	100.00%
	3.4	2.0	0.00%	0.00%	100.00%
	2.8	1.8	0.00%	0.00%	100.00%
400	2.8	1.8	0.00%	0.00%	100.00%
	2.8	1.8	0.00%	0.00%	100.00%
	3.3	2.0	0.00%	0.00%	100.00%
500	3.3	2.0	0.00%	0.00%	100.00%
	3.3	2.0	0.00%	0.00%	100.00%
	2.9	1.8	0.00%	0.00%	100.00%
800	2.9	1.8	0.00%	0.00%	100.00%
	2.9	1.8	0.00%	0.00%	100.00%
	3.2	2.0	0.00%	0.00%	100.00%
1000	3.2	2.0	0.00%	0.00%	100.00%
	3.2	2.0	0.00%	0.00%	100.00%

Table 5.4: Bipartite

Vertices	Nodes	Leaves	P_A	P_B	P_C
10	22.7	6.0	0.41%	0.41%	99.18%
	22.9	6.1	0.00%	0.00%	100.00%
	22.9	6.1	0.00%	0.00%	100.00%
20	16.1	4.7	0.00%	0.00%	100.00%
	16.1	4.7	0.00%	0.00%	100.00%
	16.1	4.7	0.00%	0.00%	100.00%
50	1.2	1.1	0.00%	0.00%	100.00%
	1.2	1.1	0.00%	0.00%	100.00%
	1.2	1.1	0.00%	0.00%	100.00%
100	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
200	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
400	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
500	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
800	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
1000	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%

Table 5.5: Dense graphs

Vertices	Invariant	Nodes	Leaves	Pruned	P_A	P_B	P_C
10	triangle	1.4	1.2	0.0	0.00%	0.00%	100.00%
10	paths	1.4	1.2	0.0	0.00%	0.00%	100.00%
10	triple	1.4	1.2	0.0	0.00%	0.00%	100.00%
50	triangle	1.0	1.0	0.0	0.00%	0.00%	0.00%
50	paths	1.0	1.0	0.0	0.00%	0.00%	0.00%
50	triple	1.0	1.0	0.0	0.00%	0.00%	0.00%
200	triangle	1.0	1.0	0.0	0.00%	0.00%	0.00%
200	paths	1.0	1.0	0.0	0.00%	0.00%	0.00%
200	triple	1.0	1.0	0.0	0.00%	0.00%	0.00%

Table 5.6: Average pruning statistics.

Vertices	Nodes	Leaves	P_A	P_B	P_C
	3.0	2.0	0.00%	0.00%	0.00%
2	3.0	2.0	0.00%	0.00%	0.00%
	3.0	2.0	0.00%	0.00%	0.00%
	6.6	3.3	0.00%	0.00%	100.00%
4	6.6	3.3	0.00%	0.00%	100.00%
	6.6	3.3	0.00%	0.00%	100.00%
	11.7	4.6	0.00%	0.00%	100.00%
8	11.7	4.6	0.00%	0.00%	100.00%
	11.7	4.6	0.00%	0.00%	100.00%
	17.0	5.5	0.00%	0.00%	100.00%
16	17.0	5.5	0.00%	0.00%	100.00%
	17.0	5.5	0.00%	0.00%	100.00%
	23.5	6.5	0.00%	0.00%	100.00%
32	23.5	6.5	0.00%	0.00%	100.00%
	23.5	6.5	0.00%	0.00%	100.00%
	1.0	1.0	0.00%	0.00%	0.00%
64	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
128	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
256	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
512	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%
1024	1.0	1.0	0.00%	0.00%	0.00%
	1.0	1.0	0.00%	0.00%	0.00%

Hypercube

Discussion of the results

Avec ces différentes classes de graphes, beaucoup de caractéristiques sautent aux yeux. Tout d'abord, on peut remarquer que les graphes aléatoires $G(n, 1/2)$, les graphes bipartites, les hypercube et les graphes denses ont un nombre de noeuds et de feuilles visités relativement faible. La plupart des graphes ont un nombre de noeuds et de feuilles visités égal à 1, ce qui signifie que l'algorithme a trouvé la solution optimale dès la fonction de refinement initial. Ce qui n'est pas le meilleur moyen de tester nos prunning mais démontre bien pour les graphes aléatoires $G(n, 1/2)$, almost all graphs can be canonically labelled in linear time by a simple refinement procedure as cited in the state of the art [6]. Pour les autres graphes, cela vient du fait qu'ils sont très structurés et que la fonction de refinement est très efficace pour trouver la solution optimale.

Cependant, là où on peut vraiment voir l'efficacité du prunning se trouve dans les autres catégories de graphes. Tout d'abord, le point le plus crucial que nous pouvons observer est que le prunning $\mathcal{P}_C$ est le plus efficace pour tous les graphes. Il est quasi à 100 pourcent pour tous les graphes qui ont subi du prunning. Ça peut s'expliquer par plusieurs raisons. La première est que nos méthodes invariants ne sont peut-être pas très bien adaptées pour les graphes que nous avons testés. En effet, les invariants que nous avons choisis sont très simples et ne sont pas très discriminants. Donc lors des comparaisons d'invariants par node, on arrive à trouver des noeuds qui ont le même invariant et donc on ne peut pas pruner. La deuxième raison est le groupe d'automorphisme est énorme, ce qui permet de faire une énorme quantité de prunning par automorphism. et comme notre prunning par automorphisme se passe avant le prunning par invariants, cela fait que le prunning par invariants est très peu efficace comparé au prunning par automorphisme.

Chapter 6

Conclusion

6.1 AI tools

Artificial intelligence tools, including ChatGPT and Claude, were used during the preparation of this thesis. Their use was limited to English-language correction and research on LaTeX functions. No artificial intelligence tool was used to generate or write the substantive content of the thesis.

With regard to the implementation, artificial intelligence tools were used for code correction and debugging, as well as for the creation of test files. The design, implementation, and validation of the algorithms and methods presented in this thesis remain the responsibility of the author.

Bibliography

[1] S. Aaronson. $P \stackrel{?}{=} NP$. In *Open problems in mathematics*, pages 1–122. Springer, 2016.

[2] A. V. Aho and J. E. Hopcroft. *The design and analysis of computer algorithms*. Pearson Education India, 1974.

[3] V. L. Arlazarov, I. I. Zuev, A. V. Uskov, and I. Faradzhev. An algorithm for the reduction of finite non-oriented graphs to canonical form. *USSR Computational Mathematics and Mathematical Physics*, 14(3), 1974.

[4] L. Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of the forty-eighth annual ACM symposium on Theory of Computing*, pages 684–697, 2016.

[5] L. Babai, G. Cooperman, L. Finkelstein, E. Luks, and Á. Seress. Fast monte carlo algorithms for permutation groups. In *Proceedings of the twenty-third annual ACM symposium on Theory of computing*, pages 90–100, 1991.

[6] L. Babai, P. Erdos, and S. M. Selkow. Random graph isomorphism. *SIaM Journal on computing*, 9(3):628–635, 1980.

[7] L. Babai, W. M. Kantor, and E. M. Luks. Computational complexity and the classification of finite simple groups. In *24th Annual Symposium on Foundations of Computer Science (Sfcs 1983)*, pages 162–171. IEEE, 1983.

[8] S. Baruah. Feasibility analysis for hpc-dag tasks. *Real-Time Systems*, 58:1–19, June 2022. DOI: 10.1007/s11241-022-09380-z.

[9] C. Berkholz, P. Bonsma, and M. Grohe. Tight lower and upper bounds for the complexity of canonical colour refinement. *Theory of Computing Systems*, 60:581–614, 2017.

[10] H. L. Bodlaender. Polynomial algorithms for graph isomorphism and chromatic index on partial k-trees. *Journal of Algorithms*, 11(4):631–643, 1990.

[11] G. Brinkmann, J. Goedgebeur, and N. Van Cleemput. The history of the generation of cubic graphs. *International Journal of Chemical Modeling*, 5(2-3):203–225, 2013.

[12] A. Cardon and M. Crochemore. Partitioning a graph in $\mathcal{O}(|A| \log_2 |V|)$. *Theoretical Computer Science*, 19(1):85–98, 1982.

[13] C. J. Colbourn and K. S. Booth. Linear time automorphism algorithms for trees, interval graphs, and planar graphs. *SIAM Journal on Computing*, 10(1):203–225, 1981.

[14] D. G. Corneil and C. C. Gotlieb. An efficient algorithm for graph isomorphism. *Journal of the ACM (JACM)*, 17(1):51–64, 1970.

[15] P. T. Darga, M. H. Liffiton, K. A. Sakallah, and I. L. Markov. Exploiting structure in symmetry detection for CNF. In *Proceedings of the 41st Annual Design Automation Conference*, pages 530–534, 2004.

[16] B. Das, M. K. Enduri, and I. V. Reddy. Polynomial-time algorithm for isomorphism of graphs with clique-width at most three. *Theoretical Computer Science*, 819:9–23, 2020.

[17] S. Datta, N. Limaye, P. Nimbhorkar, T. Thierauf, and F. Wagner. Planar graph isomorphism is in log-space. In *2009 24th Annual IEEE Conference on Computational Complexity*, pages 203–214. IEEE, 2009.

[18] R. Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, Heidelberg, 6th edition, 2025. ISBN: 978-3-662-53621-6.

[19] P. Dobsan. Pynauty: a python interface for nauty, 2023. URL: https://github.com/pdobsan/pynauty. Accessed: 2025-05-11.

[20] T. Doe and contributors. Nautygraphs.jl: a julia interface for nauty, 2023. URL: https://discourse.julialang.org/t/new-nautygraphs-jl/119029. Accessed: 2025-05-11.

[21] I. S. Filotti and J. N. Mayer. A polynomial-time algorithm for determining the isomorphism of graphs of fixed genus. In *Proceedings of the twelfth annual ACM symposium on Theory of computing*, pages 236–243, 1980.

[22] P. Foggia, C. Sansone, M. Vento, et al. A performance comparison of five algorithms for graph isomorphism. In *Proceedings of the 3rd IAPR TC-15 Workshop on Graph-based Representations in Pattern Recognition*, pages 188–199, 2001.

[23] M. Frank and M. Codish. Logic programming with graph automorphism: integrating nauty with prolog (tool description). *Theory and Practice of Logic Programming*, 16(5-6):688–702, 2016.

[24] M. Furst, J. Hopcroft, and E. Luks. Polynomial-time algorithms for permutation groups. In *21st Annual Symposium on Foundations of Computer Science (sfcs 1980)*, pages 36–41, 1980. DOI: 10.1109/SFCS.1980.34.

[25] M. R. Garey and D. S. Johnson. *Computers and intractability*, volume 29. wh freeman New York, 2002.

[26] O. Goldreich, S. Micali, and A. Wigderson. Proofs that yield nothing but their validity or all languages in NP have zero-knowledge proof systems. *Journal of the ACM (JACM)*, 38(3):690–728, 1991.

[27] M. Grohe. Structural and logical approaches to the graph isomorphism problem. In *SODA*, page 188, 2012.

[28] M. Grohe, K. Kersting, M. Mladenov, and E. Selman. Dimension reduction via colour refinement. In *Algorithms-ESA 2014: 22th Annual European Symposium, Wroclaw, Poland, September 8-10, 2014. Proceedings 21*, pages 505–516. Springer, 2014.

[29] M. Grohe and P. Schweitzer. The graph isomorphism problem. *Communications of the ACM*, 63(11):128–134, 2020.

[30] J. E. Hopcroft and J. K. Wong. Linear time algorithm for isomorphism of planar graphs (preliminary report). In *Proceedings of the Sixth Annual ACM Symposium on Theory of Computing*, STOC '74, pages 172–184, Seattle, Washington, USA. Association for Computing Machinery, 1974. ISBN: 9781450374231. DOI: 10.1145/800119.803896. URL: https://doi.org/10.1145/800119.803896.

[31] J. E. Hopcroft and J.-K. Wong. Linear time algorithm for isomorphism of planar graphs (preliminary report). In *Proceedings of the sixth annual ACM symposium on Theory of computing*, pages 172–184, 1974.

[32] R. Impagliazzo and R. Paturi. On the complexity of k-sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.

[33] A. Ivrii and O. Strichman. Exploiting isomorphic subgraphs in sat. In *2021 Formal Methods in Computer Aided Design (FMCAD)*, pages 204–211, 2021. DOI: 10.34727/2021/isbn.978-3-85448-046-4_29.

[34] T. Junttila and P. Kaski. Conflict propagation and component recursion for canonical labeling. In *International Conference on Theory and Practice of Algorithms in (Computer) Systems*, pages 151–162. Springer, 2011.

[35] S. Kiefer. *Power and limits of the Weisfeiler-Leman algorithm*. PhD thesis, Dissertation, RWTH Aachen University, 2020, 2020.

[36] S. Kiefer, I. Ponomarenko, and P. Schweitzer. The weisfeiler–leman dimension of planar graphs is at most 3. *Journal of the ACM (JACM)*, 66(6):1–31, 2019.

[37] A. Kirk. Efficiency considerations in the canonical labelling of graphs. Technical report, Technical report TR-CS-85-05, Computer Science Department, Australian, 1985.

[38] D. E. Knuth. Efficient representation of perm groups, 1991. arXiv: math/9201304 [math.GR]. URL: https://arxiv.org/abs/math/9201304.

[39] J. Köbler, S. Kuhnert, B. Laubner, and O. Verbitsky. Interval graphs: canonical representations in logspace. *SIAM Journal on Computing*, 40(5):1292–1315, 2011.

[40] J. L. López-Presa, A. F. Anta, and L. N. Chiroque. Conauto-2.0: fast isomorphism testing and automorphism group computation. *arXiv:1108.1060*, 2011.

[41] E. M. Luks. Isomorphism of graphs of bounded valence can be tested in polynomial time. *Journal of computer and system sciences*, 25(1):42–65, 1982.

[42] R. Mathon. A note on the graph isomorphism counting problem. *Information Processing Letters*, 8(3):131–136, 1979.

[43] B. McKay. Isomorph-free exhaustive generation. *Journal of Algorithms*, 26(2):306–324, 1998.

[44] B. D. McKay. Practical graph isomorphism, 1981.

[45] B. D. McKay and A. Piperno. Practical graph isomorphism, II. *Journal of symbolic computation*, 60:94–112, 2014.

[46] B. D. McKay. Description of graph6, sparse6 and digraph6 encodings. https://users.cecs.anu.edu.au/~bdm/data/formats.txt, 2022.

[47] B. D. McKay. *nauty and Traces User's Guide (Version 2.8.9)*. https://users.cecs.anu.edu.au/~bdm/nauty/nug28.pdf. Australian National University. Canberra, Australia, Aug. 2024.

[48] H. L. Morgan. The generation of a unique machine description for chemical structures a technique developed at chemical abstracts service. *Journal of chemical documentation*, 5(2):107–113, 1965.

[49] W. Myrvold and W. Kocay. Errors in graph embedding algorithms. *Journal of Computer and System Sciences*, 77(2):430–438, 2011.

[50] D. Neuen and P. Schweitzer. An exponential lower bound for individualization-refinement algorithms for graph isomorphism. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2018, pages 138–150, Los Angeles, CA, USA. Association for Computing Machinery, 2018.

[51] R. Parris and R. Read. A coding procedure for graphs. *Scientific Report. UWI/CC*, 10, 1969.

[52] A. Piperno. Search space contraction in canonical labeling of graphs. *arXiv preprint arXiv:0804.4881*, 2008.

[53] I. N. Ponomarenko. Isomorphism problem for classes of graphs closed under contractions. *Zapiski Nauchnykh Seminarov POMI*, 174:147–177, 1988.

[54] L. C. Ray and R. A. Kirsch. Finding chemical records by digital computers. *Science*, 126(3278):814–819, 1957.

[55] R. C. Read and D. G. Corneil. The graph isomorphism disease. *Journal of graph theory*, 1(4):339–363, 1977.

[56] D. C. Schmidt and L. E. Druffel. A fast backtracking algorithm to test directed graphs for isomorphism using distance matrices. *Journal of the ACM (JACM)*, 23(3):433–445, 1976.

[57] U. Schöning. Graph isomorphism is in the low hierarchy. *Journal of Computer and System Sciences*, 37(3):312–323, 1988.

[58] Á. Seress. *Permutation Group Algorithms*, volume 152 of *Cambridge Tracts in Mathematics*. Cambridge University Press, 2003.

[59] N. Shervashidze, P. Schweitzer, E. J. Van Leeuwen, K. Mehlhorn, and K. M. Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(9), 2011.

[60] M. Sipser. Introduction to the theory of computation. *ACM Sigact News*, 27(1):27–29, 1996.

[61] E. H. Sussenguth. A graph-theoretic algorithm for matching chemical structures. *Journal of Chemical Documentation*, 5(1):36–43, 1965.

[62] J. Torán. On the hardness of graph isomorphism. *SIAM Journal on Computing*, 33(5):1093–1108, 2004.

[63] S. H. Unger. Git—a heuristic program for testing pairs of directed line graphs for isomorphism. *Communications of the ACM*, 7(1):26–34, 1964.

[64] Wikipedia contributors. Orbit-stabilizer theorem — Wikipedia, the free encyclopedia, 2025.